M02 — Prácticas integrales: programar y probar APIs REST con .NET 8

Autor: JAIME GALLO

Metodología de trabajo

Cada apartado parte del diseño REST explicado en teoría y culmina en **código ejecutable y peticiones HTTP**. Se trabaja con Visual Studio, ASP.NET Core 8 Minimal API, EF Core 8 y SQL Server Express LocalDB. Los cuatro estados de referencia están en `PROYECTO/2.1` a `PROYECTO/2.4`: no son simulaciones, sino proyectos con endpoints reales probados por HTTP.

La actividad práctica no requiere operaciones Git. El alumnado construye primero la etapa 2.1, incorpora los verbos HTTP en 2.2, conecta la validación en 2.3 y documenta el mismo servicio en 2.4. Para contrastar el resultado, puede abrir por separado los proyectos de referencia. La copia no sustituye la escritura y comprensión de código.

Persistencia: la API del alumno utiliza SQL Server Express LocalDB. Los tests de integración usan EF Core InMemory únicamente para ejecutarse de forma independiente en GitHub Actions. `EnsureCreated` se emplea en desarrollo para simplificar el arranque del ejercicio; no se propone para despliegues productivos.

Punto 2.1 — Del diseño REST a los primeros endpoints GET

Proyecto comprobado: `PROYECTO/2.1-diseno-apis-rest`. El punto comienza estudiando el contrato de recursos y concluye con una ejecución verificable. Ninguna comprobación sustituye a un endpoint realmente implementado.

PASO 2.1.1 — Crear el proyecto ASP.NET Core Empty desde Visual Studio

Resultado que vas a construir. Crear el proyecto ASP.NET Core Empty desde Visual Studio. Este paso corresponde a la etapa `2.1-diseno-apis-rest` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `src/AceriaApp.Api/Modelo/Orden.cs` y localiza el concepto `Orden`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. REST identifica órdenes mediante una URI estable. ASP.NET Core no pide implementar un validador de cadenas para disponer de enrutamiento: la llamada MapGet asocia un verbo y una plantilla de ruta al manejador real. Hay que distinguir la entidad Orden, que EF Core persiste, de la representación pública que recibe el consumidor. Conocer los atributos Id, Cliente, Material, Cantidad, Estado y FechaCreacion permite justificar después por qué no todos se aceptan como entrada. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `GET /api/v1/ordenes → 200 + JSON`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.1.2 — Explorar Program.cs, Kestrel y el perfil de arranque

Resultado que vas a construir. Explorar Program.cs, Kestrel y el perfil de arranque. Este paso corresponde a la etapa `2.1-diseno-apis-rest` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `src/AceriaApp.Api/Modelo/Orden.cs` y localiza el concepto `Orden`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. REST identifica órdenes mediante una URI estable. ASP.NET Core no pide implementar un validador de cadenas para disponer de enrutamiento: la llamada MapGet asocia un verbo y una plantilla de ruta al manejador real. Hay que distinguir la entidad Orden, que EF Core persiste, de la representación pública que recibe el consumidor. Conocer los atributos Id, Cliente, Material, Cantidad, Estado y FechaCreacion permite justificar después por qué no todos se aceptan como entrada. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `GET /api/v1/ordenes/{id} → 200 / 404`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.1.3 — Dibujar los recursos y decidir las dos URIs GET

Resultado que vas a construir. Dibujar los recursos y decidir las dos URIs GET. Este paso corresponde a la etapa `2.1-diseno-apis-rest` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `src/AceriaApp.Api/Modelo/Orden.cs` y localiza el concepto `Orden`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. REST identifica órdenes mediante una URI estable. ASP.NET Core no pide implementar un validador de cadenas para disponer de enrutamiento: la llamada MapGet asocia un verbo y una plantilla de ruta al manejador real. Hay que distinguir la entidad Orden, que EF Core persiste, de la representación pública que recibe el consumidor. Conocer los atributos Id, Cliente, Material, Cantidad, Estado y FechaCreacion permite justificar después por qué no todos se aceptan como entrada. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `GET /api/v1/ordenes → 200 + JSON`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.1.4 — Crear carpeta Modelo y entidad Orden

Resultado que vas a construir. Crear carpeta Modelo y entidad Orden. Este paso corresponde a la etapa `2.1-diseno-apis-rest` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Código real que debes escribir y estudiar: **`src/AceriaApp.Api/Modelo/Orden.cs`**.

```
namespace AceriaApp.Api.Modelo;
// INICIO PASO 2.1.4
public sealed class Orden
{
    public Guid Id { get; set; } = Guid.NewGuid();
    public string Cliente { get; set; } = string.Empty;
    public string Material { get; set; } = string.Empty;
    public int Cantidad { get; set; }
    public string Estado { get; set; } = "Pendiente";
    public DateTime FechaCreacion { get; set; } = DateTime.UtcNow;
}
// FIN PASO 2.1.4
```

Lectura del código. Examina su declaración, los argumentos que ASP.NET Core enlaza, los servicios que resuelve mediante DI y la decisión que produce el resultado HTTP. Si aparece `SaveChangesAsync`, identifica cuándo se confirma la operación; si aparece `Results.Ok`, `Results.NotFound` o `Results.Problem`, explica el código y el cuerpo enviados.

Por qué se hace. REST identifica órdenes mediante una URI estable. ASP.NET Core no pide implementar un validador de cadenas para disponer de enrutamiento: la llamada MapGet asocia un verbo y una plantilla de ruta al manejador real. Hay que distinguir la entidad Orden, que EF Core persiste, de la representación pública que recibe el consumidor. Conocer los atributos Id, Cliente, Material, Cantidad,

Estado y FechaCreacion permite justificar después por qué no todos se aceptan como entrada. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `GET /api/v1/ordenes/{id} → 200 / 404`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.1.5 — Explicar ID Guid, Estado y FechaCreacion de la entidad

Resultado que vas a construir. Explicar ID Guid, Estado y FechaCreacion de la entidad. Este paso corresponde a la etapa `2.1-diseno-apis-rest` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `src/AceriaApp.Api/Modelo/Orden.cs` y localiza el concepto `Orden`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. REST identifica órdenes mediante una URI estable. ASP.NET Core no pide implementar un validador de cadenas para disponer de enrutamiento: la llamada MapGet asocia un verbo y una plantilla de ruta al manejador real. Hay que distinguir la entidad Orden, que EF Core persiste, de la representación pública que recibe el consumidor. Conocer los atributos Id, Cliente, Material, Cantidad, Estado y FechaCreacion permite justificar después por qué no todos se aceptan como entrada. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `GET /api/v1/ordenes → 200 + JSON`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.1.6 — Crear OrdenDto como contrato de salida

Resultado que vas a construir. Crear OrdenDto como contrato de salida. Este paso corresponde a la etapa `2.1-diseno-apis-rest` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Código real que debes escribir y estudiar: **`src/AceriaApp.Api/Modelo/OrdenDto.cs`**.

```
namespace AceriaApp.Api.Modelo;
// INICIO PASO 2.1.5
public sealed record OrdenDto(Guid Id, string Cliente, string Material,
    int Cantidad, string Estado, DateTime FechaCreacion)
{
    public static OrdenDto Desde(Orden x) =>
        new(x.Id,x.Cliente,x.Material,x.Cantidad,x.Estado,x.FechaCreacion);
}
// FIN PASO 2.1.5
```

Lectura del código. Examina su declaración, los argumentos que ASP.NET Core enlaza, los servicios que resuelve mediante DI y la decisión que produce el resultado HTTP. Si aparece `SaveChangesAsync`, identifica cuándo se confirma la operación; si aparece `Results.Ok`, `Results.NotFound` o `Results.Problem`, explica el código y el cuerpo enviados.

Por qué se hace. El record OrdenDto representa el JSON de salida. El método Desde transforma explícitamente la entidad a una representación. Esta distinción reduce el acoplamiento entre el esquema de la tabla y el contrato HTTP. Si se cambiara una propiedad interna de Orden no tendría por qué aparecer automáticamente en la respuesta; el método de conversión es precisamente el lugar donde decidir qué se publica. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `GET /api/v1/ordenes/{id} → 200 / 404`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.1.7 — Programar la conversión OrdenDto.Desde

Resultado que vas a construir. Programar la conversión OrdenDto.Desde. Este paso corresponde a la etapa `2.1-diseno-apis-rest` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `src/AceriaApp.Api/Modelo/OrdenDto.cs` y localiza el concepto `OrdenDto`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. El record OrdenDto representa el JSON de salida. El método Desde transforma explícitamente la entidad a una representación. Esta distinción reduce el acoplamiento entre el esquema de la tabla y el contrato HTTP. Si se cambiara una propiedad interna de Orden no tendría por qué aparecer automáticamente en la respuesta; el método de conversión es precisamente el lugar donde decidir qué se publica. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `GET /api/v1/ordenes → 200 + JSON`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.1.8 — Instalar EF Core 8 para SQL Server

Resultado que vas a construir. Instalar EF Core 8 para SQL Server. Este paso corresponde a la etapa `2.1-diseno-apis-rest` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `src/AceriaApp.Api/Modelo/OrdenDto.cs` y localiza el concepto `OrdenDto`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. El record OrdenDto representa el JSON de salida. El método Desde transforma explícitamente la entidad a una representación. Esta distinción reduce el acoplamiento entre el esquema de la tabla y el contrato HTTP. Si se cambiara una propiedad interna de Orden no tendría por qué aparecer automáticamente en la respuesta; el método de conversión es precisamente el lugar donde decidir qué se publica. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `GET /api/v1/ordenes/{id} → 200 / 404`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.1.9 — Crear AceriaDbContext y DbSet

Resultado que vas a construir. Crear AceriaDbContext y DbSet. Este paso corresponde a la etapa `2.1-diseno-apis-rest` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Código real que debes escribir y estudiar: `src/AceriaApp.Api/Datos/AceriaDbContext.cs`.

```
using AceriaApp.Api.Modelo;
using Microsoft.EntityFrameworkCore;
namespace AceriaApp.Api.Datos;
// INICIO PASO 2.1.6
public sealed class AceriaDbContext(DbContextOptions<AceriaDbContext> options) :
DbContext(options)
{
    public DbSet<Orden> Ordenes => Set<Orden>();
    protected override void OnModelCreating(ModelBuilder b)
    {
        var e=b.Entity<Orden>();
        e.HasKey(x=>x.Id);
        e.Property(x=>x.Cliente).HasMaxLength(100).IsRequired();
        e.Property(x=>x.Material).HasMaxLength(100).IsRequired();
        e.Property(x=>x.Estado).HasMaxLength(24).IsRequired();
    }
}
// FIN PASO 2.1.6
```

Lectura del código. Examina su declaración, los argumentos que ASP.NET Core enlaza, los servicios que resuelve mediante DI y la decisión que produce el resultado HTTP. Si aparece `SaveChangesAsync`, identifica cuándo se confirma la operación; si aparece `Results.Ok`, `Results.NotFound` o `Results.Problem`, explica el código y el cuerpo enviados.

Por qué se hace. El record OrdenDto representa el JSON de salida. El método Desde transforma explícitamente la entidad a una representación. Esta distinción reduce el acoplamiento entre el esquema de la tabla y el contrato HTTP. Si se cambiara una propiedad interna de Orden no tendría por qué aparecer automáticamente en la respuesta; el método de conversión es precisamente el lugar donde decidir qué se publica. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `GET /api/v1/ordenes → 200 + JSON`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.1.10 — Configurar OnModelCreating, clave y longitudes

Resultado que vas a construir. Configurar OnModelCreating, clave y longitudes. Este paso corresponde a la etapa `2.1-diseno-apis-rest` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `src/AceriaApp.Api/Modelo/OrdenDto.cs` y localiza el concepto `OrdenDto`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. El record OrdenDto representa el JSON de salida. El método Desde transforma explícitamente la entidad a una representación. Esta distinción reduce el acoplamiento entre el esquema de la tabla y el contrato HTTP. Si se cambiara una propiedad interna de Orden no tendría por qué aparecer automáticamente en la respuesta; el método de conversión es precisamente el lugar donde decidir qué se publica. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `GET /api/v1/ordenes/{id} → 200 / 404`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.1.11 — Configurar la cadena de conexión a LocalDB

Resultado que vas a construir. Configurar la cadena de conexión a LocalDB. Este paso corresponde a la etapa `2.1-diseno-apis-rest` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `src/AceriaApp.Api/Datos/AceriaDbContext.cs` y localiza el concepto `AceriaDbContext`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. El contexto contiene DbSet y las restricciones de las columnas. La API se ejecuta sobre SQL Server Express LocalDB en el ordenador del alumno. Registrar AddDbContext permite que los endpoints reciban el contexto mediante inyección de dependencias: no hay que crear el contexto con new dentro de cada manejador. En tests se sustituye solamente el proveedor por InMemory para evitar depender de Windows. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `GET /api/v1/ordenes → 200 + JSON`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.1.12 — Registrar AceriaDbContext mediante AddDbContext

Resultado que vas a construir. Registrar AceriaDbContext mediante AddDbContext. Este paso corresponde a la etapa `2.1-diseno-apis-rest` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `src/AceriaApp.Api/Datos/AceriaDbContext.cs` y localiza el concepto `AceriaDbContext`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. El contexto contiene DbSet y las restricciones de las columnas. La API se ejecuta sobre SQL Server Express LocalDB en el ordenador del alumno. Registrar AddDbContext permite que los endpoints reciban el contexto mediante inyección de dependencias: no hay que crear el contexto con new dentro de cada manejador. En tests se sustituye solamente el proveedor por InMemory para evitar depender de Windows. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `GET /api/v1/ordenes/{id} → 200 / 404`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.1.13 — Crear la base didáctica solo en Development

Resultado que vas a construir. Crear la base didáctica solo en Development. Este paso corresponde a la etapa `2.1-diseno-apis-rest` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `src/AceriaApp.Api/Datos/AceriaDbContext.cs` y localiza el concepto `AceriaDbContext`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. El contexto contiene DbSet y las restricciones de las columnas. La API se ejecuta sobre SQL Server Express LocalDB en el ordenador del alumno. Registrar AddDbContext permite que los endpoints reciban el contexto mediante inyección de dependencias: no hay que crear el contexto con

new dentro de cada manejador. En tests se sustituye solamente el proveedor por InMemory para evitar depender de Windows. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `GET /api/v1/ordenes → 200 + JSON`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.1.14 — Escribir el endpoint GET de colección

Resultado que vas a construir. Escribir el endpoint GET de colección. Este paso corresponde a la etapa `2.1-diseno-apis-rest` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Código real que debes escribir y estudiar: **`src/AceriaApp.Api/Program.cs`**.

```
using AceriaApp.Api.Datos;
using AceriaApp.Api.Modelo;
using Microsoft.EntityFrameworkCore;

var builder=WebApplication.CreateBuilder(args);
// INICIO PASO 2.1.7: SQL Server Express LocalDB, no SQLite.
builder.Services.AddDbContext<AceriaDbContext>(o=>
    o.UseSqlServer(builder.Configuration.GetConnectionString("AceriaApp")));

var app=builder.Build();
// EnsureCreated se utiliza SOLO como ayuda didáctica en Development.
// Para un sistema productivo se utilizarían migraciones controladas.
if(app.Environment.IsDevelopment()&&!
app.Configuration.GetValue<bool>("SkipDatabaseInitialization"))
{
    using var scope=app.Services.CreateScope();
    await
scope.ServiceProvider.GetRequiredService<AceriaDbContext>().Database.EnsureCreatedAsync();
}

// INICIO PASO 2.1.8: colección de recursos, consulta real con EF Core.
app.MapGet("/api/v1/ordenes",async (AceriaDbContext db,CancellationToken ct)=>
{
    var ordenes=await db.Ordenes.AsNoTracking().OrderBy(x=>x.FechaCreacion).ToListAsync(ct);
    return Results.Ok(ordenes.Select(OrdenDto.Desde));
})
.WithName("ListarOrdenes").WithTags("Ordenes")

;
```

```
// INICIO PASO 2.1.9: recurso individual.
app.MapGet("/api/v1/ordenes/{id:guid}",async (Guid id,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.AsNoTracking().FirstOrDefaultAsync(x=>x.Id==id);
    IResult response=orden is null ? Results.NotFound() : Results.Ok(OrdenDto.Desde(orden));
    return response;
})
.WithName("ObtenerOrdenPorId").WithTags("Ordenes")

;

app.Run();
public partial class Program {} // permite WebApplicationFactory en tests
```

Lectura del código. Examina su declaración, los argumentos que ASP.NET Core enlaza, los servicios que resuelve mediante DI y la decisión que produce el resultado HTTP. Si aparece `SaveChangesAsync`, identifica cuándo se confirma la operación; si aparece `Results.Ok`, `Results.NotFound` o `Results.Problem`, explica el código y el cuerpo enviados.

Por qué se hace. El contexto contiene DbSet y las restricciones de las columnas. La API se ejecuta sobre SQL Server Express LocalDB en el ordenador del alumno. Registrar AddDbContext permite que los endpoints reciban el contexto mediante inyección de dependencias: no hay que crear el contexto con new dentro de cada manejador. En tests se sustituye solamente el proveedor por InMemory para evitar depender de Windows. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `GET /api/v1/ordenes/{id} → 200 / 404`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.1.15 — Utilizar AsNoTracking y proyectar a DTO

Resultado que vas a construir. Utilizar AsNoTracking y proyectar a DTO. Este paso corresponde a la etapa `2.1-diseno-apis-rest` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `src/AceriaApp.Api/Datos/AceriaDbContext.cs` y localiza el concepto `AceriaDbContext`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. El contexto contiene DbSet y las restricciones de las columnas. La API se ejecuta sobre SQL Server Express LocalDB en el ordenador del alumno. Registrar AddDbContext permite que los endpoints reciban el contexto mediante inyección de dependencias: no hay que crear el contexto con new dentro de cada manejador. En tests se sustituye solamente el proveedor por InMemory para evitar

depender de Windows. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `GET /api/v1/ordenes → 200 + JSON`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.1.16 — Ejecutar GET y verificar 200 con lista vacía

Resultado que vas a construir. Ejecutar GET y verificar 200 con lista vacía. Este paso corresponde a la etapa `2.1-diseno-apis-rest` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `src/AceriaApp.Api/Program.cs` y localiza el concepto `MapGet`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. El manejador obtiene AceriaDbContext, usa AsNoTracking para consultar sin seguimiento y devuelve Results.Ok con una secuencia de DTOs. En una colección vacía se responde 200 y una lista JSON vacía, pues la colección como recurso existe aunque no contenga elementos. La respuesta debe inspeccionarse desde AceriaApp.http, no deducirse de que el código compile. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `GET /api/v1/ordenes/{id} → 200 / 404`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.1.17 — Escribir el GET individual con

Resultado que vas a construir. Escribir el GET individual con {id:guid}. Este paso corresponde a la etapa `2.1-diseno-apis-rest` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Código real que debes escribir y estudiar: `src/AceriaApp.Api/Program.cs`.

```
using AceriaApp.Api.Datos;
using AceriaApp.Api.Modelo;
using Microsoft.EntityFrameworkCore;

var builder=WebApplication.CreateBuilder(args);
// INICIO PASO 2.1.7: SQL Server Express LocalDB, no SQLite.
builder.Services.AddDbContext<AceriaDbContext>(o=>
    o.UseSqlServer(builder.Configuration.GetConnectionString("AceriaApp")));

var app=builder.Build();
// EnsureCreated se utiliza SOLO como ayuda didáctica en Development.
// Para un sistema productivo se utilizarían migraciones controladas.
if(app.Environment.IsDevelopment()&&!
app.Configuration.GetValue<bool>("SkipDatabaseInitialization"))
{
    using var scope=app.Services.CreateScope();
    await
scope.ServiceProvider.GetRequiredService<AceriaDbContext>().Database.EnsureCreatedAsync();
}

// INICIO PASO 2.1.8: colección de recursos, consulta real con EF Core.
app.MapGet("/api/v1/ordenes",async (AceriaDbContext db,CancellationToken ct)=>
{
    var ordenes=await db.Ordenes.AsNoTracking().OrderBy(x=>x.FechaCreacion).ToListAsync(ct);
    return Results.Ok(ordenes.Select(OrdenDto.Desde));
})
.WithName("ListarOrdenes").WithTags("Ordenes")

;
// INICIO PASO 2.1.9: recurso individual.
app.MapGet("/api/v1/ordenes/{id:guid}",async (Guid id,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.AsNoTracking().FirstOrDefaultAsync(x=>x.Id==id);
    IResult response=orden is null ? Results.NotFound() : Results.Ok(OrdenDto.Desde(orden));
    return response;
})
.WithName("ObtenerOrdenPorId").WithTags("Ordenes")

;

app.Run();
public partial class Program {} // permite WebApplicationFactory en tests
```

Lectura del código. Examina su declaración, los argumentos que ASP.NET Core enlaza, los servicios que resuelve mediante DI y la decisión que produce el resultado HTTP. Si aparece `SaveChangesAsync`, identifica cuándo se confirma la operación; si aparece `Results.Ok`, `Results.NotFound` o `Results.Problem`, explica el código y el cuerpo enviados.

Por qué se hace. El manejador obtiene AceriaDbContext, usa AsNoTracking para consultar sin seguimiento y devuelve Results.Ok con una secuencia de DTOs. En una colección vacía se responde 200 y una lista JSON vacía, pues la colección como recurso existe aunque no contenga elementos. La respuesta debe inspeccionarse desde AceriaApp.http, no deducirse de que el código compile. En esta

actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `GET /api/v1/ordenes → 200 + JSON`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.1.18 — Devolver NotFound cuando no existe la orden

Resultado que vas a construir. Devolver NotFound cuando no existe la orden. Este paso corresponde a la etapa `2.1-diseno-apis-rest` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `src/AceriaApp.Api/Program.cs` y localiza el concepto `MapGet`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. El manejador obtiene AceriaDbContext, usa AsNoTracking para consultar sin seguimiento y devuelve Results.Ok con una secuencia de DTOs. En una colección vacía se responde 200 y una lista JSON vacía, pues la colección como recurso existe aunque no contenga elementos. La respuesta debe inspeccionarse desde AceriaApp.http, no deducirse de que el código compile. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `GET /api/v1/ordenes/{id} → 200 / 404`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.1.19 — Comprobar diferencias entre ruta incorrecta y recurso ausente

Resultado que vas a construir. Comprobar diferencias entre ruta incorrecta y recurso ausente. Este paso corresponde a la etapa `2.1-diseno-apis-rest` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `src/AceriaApp.Api/Program.cs` y localiza el concepto `MapGet`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. El manejador obtiene AceriaDbContext, usa AsNoTracking para consultar sin seguimiento y devuelve Results.Ok con una secuencia de DTOs. En una colección vacía se responde 200 y una lista JSON vacía, pues la colección como recurso existe aunque no contenga elementos. La respuesta debe inspeccionarse desde AceriaApp.http, no deducirse de que el código compile. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `GET /api/v1/ordenes → 200 + JSON`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.1.20 — Crear AceriaApp.http y enviar peticiones desde Visual Studio

Resultado que vas a construir. Crear AceriaApp.http y enviar peticiones desde Visual Studio. Este paso corresponde a la etapa `2.1-diseno-apis-rest` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `src/AceriaApp.Api/Program.cs` y localiza el concepto `MapGet`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. El manejador obtiene AceriaDbContext, usa AsNoTracking para consultar sin seguimiento y devuelve Results.Ok con una secuencia de DTOs. En una colección vacía se responde 200 y una lista JSON vacía, pues la colección como recurso existe aunque no contenga elementos. La respuesta debe inspeccionarse desde AceriaApp.http, no deducirse de que el código compile. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `GET /api/v1/ordenes/{id} → 200 / 404`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.1.21 — Crear el proyecto xUnit de pruebas HTTP

Resultado que vas a construir. Crear el proyecto xUnit de pruebas HTTP. Este paso corresponde a la etapa `2.1-diseno-apis-rest` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Código real que debes escribir y estudiar: **`tests/AceriaApp.Api.Tests/GetTests.cs`**.

```
using System.Net;
using System.Net.Http.Json;
using AceriaApp.Api.Datos;
using AceriaApp.Api.Modelo;
using Microsoft.Extensions.DependencyInjection;
using Xunit;
namespace AceriaApp.Api.Tests;
// INICIO PASO 2.1.13: dos peticiones HTTP, código y DTO.
public class GetTests
{
    [Fact] public async Task ListaVacia_Es200()
    {
        using var f=new ApiFactory();
        var r=await f.CreateClient().GetAsync("/api/v1/ordenes");
        Assert.Equal(HttpStatusCode.OK,r.StatusCode);
    }
    [Fact] public async Task RecursoInexistente_Es404()
    {
        using var f=new ApiFactory();
        var r=await f.CreateClient().GetAsync("/api/v1/ordenes/"+Guid.NewGuid());
        Assert.Equal(HttpStatusCode.NotFound,r.StatusCode);
    }
    [Fact] public async Task RecursoExistente_ExponeDto()
    {
        using var f=new ApiFactory();
        var id=Guid.NewGuid();
        using(var scope=f.Services.CreateScope())
        {
            var db=scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
            db.Ordenes.Add(new Orden{Id=id,Cliente="ACX-001",Material="CHAPA",Cantidad=10});
            await db.SaveChangesAsync();
        }
        var r=await f.CreateClient().GetAsync("/api/v1/ordenes/"+id);
        var dto=await r.Content.ReadFromJsonAsync<OrdenDto>();
        Assert.Equal(HttpStatusCode.OK,r.StatusCode);
        Assert.Equal(id,dto?.Id);
    }
}
```

Lectura del código. Examina su declaración, los argumentos que ASP.NET Core enlaza, los servicios que resuelve mediante DI y la decisión que produce el resultado HTTP. Si aparece `SaveChangesAsync`, identifica cuándo se confirma la operación; si aparece `Results.Ok`, `Results.NotFound` o `Results.Problem`, explica el código y el cuerpo enviados.

Por qué se hace. La plantilla {id:guid} restringe el parámetro de ruta a GUID válidos. El manejador consulta una orden específica y decide entre Results.Ok y Results.NotFound. Una petición a un GUID inexistente no debería convertirse en 500 ni en una respuesta 200 con contenido vacío. WebApplicationFactory inicia la aplicación en memoria y permite comprobar 200 y 404 sobre los endpoints auténticos. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `GET /api/v1/ordenes → 200 + JSON`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.1.22 — Configurar WebApplicationFactory y base aislada para tests

Resultado que vas a construir. Configurar WebApplicationFactory y base aislada para tests. Este paso corresponde a la etapa `2.1-diseno-apis-rest` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `tests/AceriaApp.Api.Tests/GetTests.cs` y localiza el concepto `ObtenerOrdenPorId`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. La plantilla {id:guid} restringe el parámetro de ruta a GUID válidos. El manejador consulta una orden específica y decide entre Results.Ok y Results.NotFound. Una petición a un GUID inexistente no debería convertirse en 500 ni en una respuesta 200 con contenido vacío. WebApplicationFactory inicia la aplicación en memoria y permite comprobar 200 y 404 sobre los endpoints auténticos. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `GET /api/v1/ordenes/{id} → 200 / 404`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.1.23 — Probar 200, 404 y DTO mediante GetTests

Resultado que vas a construir. Probar 200, 404 y DTO mediante GetTests. Este paso corresponde a la etapa `2.1-diseno-apis-rest` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Código real que debes escribir y estudiar: **`tests/AceriaApp.Api.Tests/GetTests.cs`**.

```
using System.Net;
using System.Net.Http.Json;
using AceriaApp.Api.Datos;
using AceriaApp.Api.Modelo;
using Microsoft.Extensions.DependencyInjection;
using Xunit;
namespace AceriaApp.Api.Tests;
// INICIO PASO 2.1.13: dos peticiones HTTP, código y DTO.
public class GetTests
{
    [Fact] public async Task ListaVacia_Es200()
    {
        using var f=new ApiFactory();
        var r=await f.CreateClient().GetAsync("/api/v1/ordenes");
        Assert.Equal(HttpStatusCode.OK,r.StatusCode);
    }
    [Fact] public async Task RecursoInexistente_Es404()
    {
        using var f=new ApiFactory();
        var r=await f.CreateClient().GetAsync("/api/v1/ordenes/"+Guid.NewGuid());
        Assert.Equal(HttpStatusCode.NotFound,r.StatusCode);
    }
    [Fact] public async Task RecursoExistente_ExponeDto()
    {
        using var f=new ApiFactory();
        var id=Guid.NewGuid();
        using(var scope=f.Services.CreateScope())
        {
            var db=scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
            db.Ordenes.Add(new Orden{Id=id,Cliente="ACX-001",Material="CHAPA",Cantidad=10});
            await db.SaveChangesAsync();
        }
        var r=await f.CreateClient().GetAsync("/api/v1/ordenes/"+id);
        var dto=await r.Content.ReadFromJsonAsync<OrdenDto>();
        Assert.Equal(HttpStatusCode.OK,r.StatusCode);
        Assert.Equal(id,dto?.Id);
    }
}
```

Lectura del código. Examina su declaración, los argumentos que ASP.NET Core enlaza, los servicios que resuelve mediante DI y la decisión que produce el resultado HTTP. Si aparece `SaveChangesAsync`, identifica cuándo se confirma la operación; si aparece `Results.Ok`, `Results.NotFound` o `Results.Problem`, explica el código y el cuerpo enviados.

Por qué se hace. La plantilla {id:guid} restringe el parámetro de ruta a GUID válidos. El manejador consulta una orden específica y decide entre Results.Ok y Results.NotFound. Una petición a un GUID inexistente no debería convertirse en 500 ni en una respuesta 200 con contenido vacío. WebApplicationFactory inicia la aplicación en memoria y permite comprobar 200 y 404 sobre los

endpoints auténticos. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `GET /api/v1/ordenes → 200 + JSON`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.1.24 — Reto: incorporar consulta por cliente y explicar su URI

Resultado que vas a construir. Reto: incorporar consulta por cliente y explicar su URI. Este paso corresponde a la etapa `2.1-diseno-apis-rest` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `tests/AceriaApp.Api.Tests/GetTests.cs` y localiza el concepto `ObtenerOrdenPorId`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. La plantilla {id:guid} restringe el parámetro de ruta a GUID válidos. El manejador consulta una orden específica y decide entre Results.Ok y Results.NotFound. Una petición a un GUID inexistente no debería convertirse en 500 ni en una respuesta 200 con contenido vacío. WebApplicationFactory inicia la aplicación en memoria y permite comprobar 200 y 404 sobre los endpoints auténticos. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `GET /api/v1/ordenes/{id} → 200 / 404`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

Matriz de comprobación del punto

Comportamiento	Resultado esperado
`GET /api/v1/ordenes`	200 + JSON
`GET /api/v1/ordenes/{id}`	200 / 404

Reto integrador. Reconstruye sin consultar la solución el último endpoint o validación practicada y añade una prueba HTTP que demuestre su resultado. Describe el recurso, el verbo, el contrato y el código devuelto. Si no puedes reproducir una respuesta, vuelve al primer paso que introdujo el comportamiento.

Punto 2.2 — Recursos, verbos, persistencia y respuestas HTTP

Proyecto comprobado: `PROYECTO/2.2-recursos-verbos-codigos`. El punto comienza estudiando el contrato de recursos y concluye con una ejecución verificable. Ninguna comprobación sustituye a un endpoint realmente implementado.

PASO 2.2.1 — Recuperar el estado GET funcional sin modificarlo

Resultado que vas a construir. Recuperar el estado GET funcional sin modificarlo. Este paso corresponde a la etapa `2.2-recursos-verbos-codigos` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `src/AceriaApp.Api/Contratos/ContratosOrden.cs` y localiza el concepto `CrearOrdenRequest`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. El servidor debe recibir solamente Cliente, Material y Cantidad en CrearOrdenRequest. El ID, el Estado y la FechaCreacion pertenecen al control interno del servicio. Por eso no reutilizamos Orden como cuerpo de entrada. Este diseño protege el contrato público y delimita claramente los datos que el cliente puede modificar. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `POST /api/v1/ordenes → 201 + Location`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.2.2 — Diseñar la tabla verbo, URI, petición y respuesta

Resultado que vas a construir. Diseñar la tabla verbo, URI, petición y respuesta. Este paso corresponde a la etapa `2.2-recursos-verbos-codigos` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `src/AceriaApp.Api/Contratos/ContratosOrden.cs` y localiza el concepto `CrearOrdenRequest`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. El servidor debe recibir solamente Cliente, Material y Cantidad en CrearOrdenRequest. El ID, el Estado y la FechaCreacion pertenecen al control interno del servicio. Por eso no reutilizamos Orden como cuerpo de entrada. Este diseño protege el contrato público y delimita claramente los datos que el cliente puede modificar. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `PUT /{id} → 204`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.2.3 — Separar contrato de entrada y DTO de salida

Resultado que vas a construir. Separar contrato de entrada y DTO de salida. Este paso corresponde a la etapa `2.2-recursos-verbos-codigos` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `src/AceriaApp.Api/Contratos/ContratosOrden.cs` y localiza el concepto `CrearOrdenRequest`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. El servidor debe recibir solamente Cliente, Material y Cantidad en CrearOrdenRequest. El ID, el Estado y la FechaCreacion pertenecen al control interno del servicio. Por eso no reutilizamos Orden como cuerpo de entrada. Este diseño protege el contrato público y delimita claramente los datos que el cliente puede modificar. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `PATCH /{id}/cantidad → 200`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.2.4 — Crear CrearOrdenRequest y ActualizarCantidadRequest

Resultado que vas a construir. Crear CrearOrdenRequest y ActualizarCantidadRequest. Este paso corresponde a la etapa `2.2-recursos-verbos-codigos` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Código real que debes escribir y estudiar: **`src/AceriaApp.Api/Contratos/ContratosOrden.cs`**.

```
namespace AceriaApp.Api.Contratos;
// INICIO PASO 2.2.4
public sealed record CrearOrdenRequest(string Cliente,string Material,int Cantidad);
public sealed record ActualizarCantidadRequest(int Cantidad);
// FIN PASO 2.2.4
```

Lectura del código. Examina su declaración, los argumentos que ASP.NET Core enlaza, los servicios que resuelve mediante DI y la decisión que produce el resultado HTTP. Si aparece `SaveChangesAsync`, identifica cuándo se confirma la operación; si aparece `Results.Ok`, `Results.NotFound` o `Results.Problem`, explica el código y el cuerpo enviados.

Por qué se hace. El servidor debe recibir solamente Cliente, Material y Cantidad en CrearOrdenRequest. El ID, el Estado y la FechaCreacion pertenecen al control interno del servicio. Por eso no reutilizamos Orden como cuerpo de entrada. Este diseño protege el contrato público y delimita claramente los datos que el cliente puede modificar. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `DELETE /{id} → 204 / 404`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.2.5 — Programar MapPost y recibir JSON de una orden

Resultado que vas a construir. Programar MapPost y recibir JSON de una orden. Este paso corresponde a la etapa `2.2-recursos-verbos-codigos` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Código real que debes escribir y estudiar: **`src/AceriaApp.Api/Program.cs`**.

```
// INICIO PASO 2.2.5: crear una orden, persistirla, devolver 201 + Location.
app.MapPost("/api/v1/ordenes",async (CrearOrdenRequest request,AceriaDbContext db)=>
{
    if(string.IsNullOrWhiteSpace(request.Cliente)||string.IsNullOrWhiteSpace(request.Material)||
request.Cantidad<=0)
        return Results.BadRequest(new {error="Cliente, material y cantidad son obligatorios."});
    if(request.Cantidad>500)
        return Results.Problem(title:"El taller solo admite lotes de hasta 500.",
            statusCode:StatusCodes.Status422UnprocessableEntity);
    var orden=new
Orden{Cliente=request.Cliente,Material=request.Material,Cantidad=request.Cantidad};
    db.Ordenes.Add(orden);
    await db.SaveChangesAsync();
    return Results.CreatedAtRoute("ObtenerOrdenPorId",new{id=orden.Id},OrdenDto.Desde(orden));
})
.WithName("CrearOrden").WithTags("Ordenes")


;
```

Lectura del código. Examina su declaración, los argumentos que ASP.NET Core enlaza, los servicios que resuelve mediante DI y la decisión que produce el resultado HTTP. Si aparece `SaveChangesAsync`, identifica cuándo se confirma la operación; si aparece `Results.Ok`, `Results.NotFound` o `Results.Problem`, explica el código y el cuerpo enviados.

Por qué se hace. El servidor debe recibir solamente Cliente, Material y Cantidad en CrearOrdenRequest. El ID, el Estado y la FechaCreacion pertenecen al control interno del servicio. Por eso no reutilizamos Orden como cuerpo de entrada. Este diseño protege el contrato público y delimita claramente los datos que el cliente puede modificar. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `POST /{id}/finalizar → 200 / 409`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.2.6 — Persistir la orden con Add y SaveChangesAsync

Resultado que vas a construir. Persistir la orden con Add y SaveChangesAsync. Este paso corresponde a la etapa `2.2-recursos-verbos-codigos` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `src/AceriaApp.Api/Contratos/ContratosOrden.cs` y localiza el concepto `MapPost`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. MapPost convierte un cuerpo JSON en CrearOrdenRequest. Al construir Orden y añadirla al DbContext todavía no se ha confirmado una inserción; SaveChangesAsync ejecuta la escritura. CreatedAtRoute utiliza el nombre ObtenerOrdenPorId para incluir un vínculo al recurso y devolver 201 Created. En prácticas se enviará POST y se seguirá Location con GET para demostrar persistencia real. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `Cantidad 501 → 422`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.2.7 — Devolver 201 y Location con CreatedAtRoute

Resultado que vas a construir. Devolver 201 y Location con CreatedAtRoute. Este paso corresponde a la etapa `2.2-recursos-verbos-codigos` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Código real que debes escribir y estudiar: **`src/AceriaApp.Api/Program.cs`**.

```
// INICIO PASO 2.2.5: crear una orden, persistirla, devolver 201 + Location.
app.MapPost("/api/v1/ordenes",async (CrearOrdenRequest request,AceriaDbContext db)=>
{
    if(string.IsNullOrWhiteSpace(request.Cliente)||string.IsNullOrWhiteSpace(request.Material)||
request.Cantidad<=0)
        return Results.BadRequest(new {error="Cliente, material y cantidad son obligatorios."});
    if(request.Cantidad>500)
        return Results.Problem(title:"El taller solo admite lotes de hasta 500.",
            statusCode:StatusCodes.Status422UnprocessableEntity);
    var orden=new
Orden{Cliente=request.Cliente,Material=request.Material,Cantidad=request.Cantidad};
    db.Ordenes.Add(orden);
    await db.SaveChangesAsync();
    return Results.CreatedAtRoute("ObtenerOrdenPorId",new{id=orden.Id},OrdenDto.Desde(orden));
})
.WithName("CrearOrden").WithTags("Ordenes")


;
```

Lectura del código. Examina su declaración, los argumentos que ASP.NET Core enlaza, los servicios que resuelve mediante DI y la decisión que produce el resultado HTTP. Si aparece `SaveChangesAsync`, identifica cuándo se confirma la operación; si aparece `Results.Ok`, `Results.NotFound` o `Results.Problem`, explica el código y el cuerpo enviados.

Por qué se hace. MapPost convierte un cuerpo JSON en CrearOrdenRequest. Al construir Orden y añadirla al DbContext todavía no se ha confirmado una inserción; SaveChangesAsync ejecuta la escritura. CreatedAtRoute utiliza el nombre ObtenerOrdenPorId para incluir un vínculo al recurso y devolver 201 Created. En prácticas se enviará POST y se seguirá Location con GET para demostrar persistencia real. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `POST /api/v1/ordenes → 201 + Location`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.2.8 — Enviar POST y consultar por GET la orden creada

Resultado que vas a construir. Enviar POST y consultar por GET la orden creada. Este paso corresponde a la etapa `2.2-recursos-verbos-codigos` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `src/AceriaApp.Api/Contratos/ContratosOrden.cs` y localiza el concepto `MapPost`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. MapPost convierte un cuerpo JSON en CrearOrdenRequest. Al construir Orden y añadirla al DbContext todavía no se ha confirmado una inserción; SaveChangesAsync ejecuta la escritura. CreatedAtRoute utiliza el nombre ObtenerOrdenPorId para incluir un vínculo al recurso y devolver 201 Created. En prácticas se enviará POST y se seguirá Location con GET para demostrar persistencia real. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `PUT /{id} → 204`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.2.9 — Rechazar provisionalmente peticiones sin datos

Resultado que vas a construir. Rechazar provisionalmente peticiones sin datos. Este paso corresponde a la etapa `2.2-recursos-verbos-codigos` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `src/AceriaApp.Api/Program.cs` y localiza el concepto `MapPost`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. MapPost convierte un cuerpo JSON en CrearOrdenRequest. Al construir Orden y añadirla al DbContext todavía no se ha confirmado una inserción; SaveChangesAsync ejecuta la escritura. CreatedAtRoute utiliza el nombre ObtenerOrdenPorId para incluir un vínculo al recurso y devolver 201 Created. En prácticas se enviará POST y se seguirá Location con GET para demostrar persistencia real. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `PATCH /{id}/cantidad → 200`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.2.10 — Comprobar la política del taller y respuesta 422

Resultado que vas a construir. Comprobar la política del taller y respuesta 422. Este paso corresponde a la etapa `2.2-recursos-verbos-codigos` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `src/AceriaApp.Api/Program.cs` y localiza el concepto `MapPost`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. MapPost convierte un cuerpo JSON en CrearOrdenRequest. Al construir Orden y añadirla al DbContext todavía no se ha confirmado una inserción; SaveChangesAsync ejecuta la escritura. CreatedAtRoute utiliza el nombre ObtenerOrdenPorId para incluir un vínculo al recurso y devolver 201 Created. En prácticas se enviará POST y se seguirá Location con GET para demostrar persistencia real. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `DELETE /{id} → 204 / 404`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.2.11 — Programar MapPut con identificador Guid

Resultado que vas a construir. Programar MapPut con identificador Guid. Este paso corresponde a la etapa `2.2-recursos-verbos-codigos` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Código real que debes escribir y estudiar: **`src/AceriaApp.Api/Program.cs`**.

```
// INICIO PASO 2.2.5: crear una orden, persistirla, devolver 201 + Location.
app.MapPost("/api/v1/ordenes",async (CrearOrdenRequest request,AceriaDbContext db)=>
{
    if(string.IsNullOrWhiteSpace(request.Cliente)||string.IsNullOrWhiteSpace(request.Material)||
request.Cantidad<=0)
        return Results.BadRequest(new {error="Cliente, material y cantidad son obligatorios."});
    if(request.Cantidad>500)
        return Results.Problem(title:"El taller solo admite lotes de hasta 500.",
            statusCode:StatusCodes.Status422UnprocessableEntity);
    var orden=new
Orden{Cliente=request.Cliente,Material=request.Material,Cantidad=request.Cantidad};
    db.Ordenes.Add(orden);
    await db.SaveChangesAsync();
    return Results.CreatedAtRoute("ObtenerOrdenPorId",new{id=orden.Id},OrdenDto.Desde(orden));
})
.WithName("CrearOrden").WithTags("Ordenes")


;
```

Lectura del código. Examina su declaración, los argumentos que ASP.NET Core enlaza, los servicios que resuelve mediante DI y la decisión que produce el resultado HTTP. Si aparece `SaveChangesAsync`, identifica cuándo se confirma la operación; si aparece `Results.Ok`, `Results.NotFound` o `Results.Problem`, explica el código y el cuerpo enviados.

Por qué se hace. El ejemplo PUT reemplaza los campos editables de una orden ya existente, conservando los valores que controla el servidor. No debe inventar una nueva orden cuando el identificador no existe: en nuestro contrato responde 404. Cuando una orden ya está finalizada la regla del dominio impide editarla y se comunica un conflicto 409. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `POST /{id}/finalizar → 200 / 409`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.2.12 — Modificar Cliente, Material y Cantidad con PUT

Resultado que vas a construir. Modificar Cliente, Material y Cantidad con PUT. Este paso corresponde a la etapa `2.2-recursos-verbos-codigos` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `src/AceriaApp.Api/Program.cs` y localiza el concepto `MapPut`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. El ejemplo PUT reemplaza los campos editables de una orden ya existente, conservando los valores que controla el servidor. No debe inventar una nueva orden cuando el identificador no existe: en nuestro contrato responde 404. Cuando una orden ya está finalizada la regla del dominio impide editarla y se comunica un conflicto 409. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `Cantidad 501 → 422`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.2.13 — Comprobar 204 y consultar por GET los cambios

Resultado que vas a construir. Comprobar 204 y consultar por GET los cambios. Este paso corresponde a la etapa `2.2-recursos-verbos-codigos` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `src/AceriaApp.Api/Program.cs` y localiza el concepto `MapPut`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. El ejemplo PUT reemplaza los campos editables de una orden ya existente, conservando los valores que controla el servidor. No debe inventar una nueva orden cuando el identificador no existe: en nuestro contrato responde 404. Cuando una orden ya está finalizada la regla

del dominio impide editarla y se comunica un conflicto 409. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `POST /api/v1/ordenes → 201 + Location`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.2.14 — Programar MapPatch para cambiar solo Cantidad

Resultado que vas a construir. Programar MapPatch para cambiar solo Cantidad. Este paso corresponde a la etapa `2.2-recursos-verbos-codigos` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Código real que debes escribir y estudiar: **`src/AceriaApp.Api/Program.cs`**.

```
// INICIO PASO 2.2.5: crear una orden, persistirla, devolver 201 + Location.
app.MapPost("/api/v1/ordenes",async (CrearOrdenRequest request,AceriaDbContext db)=>
{
    if(string.IsNullOrWhiteSpace(request.Cliente)||string.IsNullOrWhiteSpace(request.Material)||
request.Cantidad<=0)
        return Results.BadRequest(new {error="Cliente, material y cantidad son obligatorios."});
    if(request.Cantidad>500)
        return Results.Problem(title:"El taller solo admite lotes de hasta 500.",
            statusCode:StatusCodes.Status422UnprocessableEntity);
    var orden=new
Orden{Cliente=request.Cliente,Material=request.Material,Cantidad=request.Cantidad};
    db.Ordenes.Add(orden);
    await db.SaveChangesAsync();
    return Results.CreatedAtRoute("ObtenerOrdenPorId",new{id=orden.Id},OrdenDto.Desde(orden));
})
.WithName("CrearOrden").WithTags("Ordenes")


;
```

Lectura del código. Examina su declaración, los argumentos que ASP.NET Core enlaza, los servicios que resuelve mediante DI y la decisión que produce el resultado HTTP. Si aparece `SaveChangesAsync`, identifica cuándo se confirma la operación; si aparece `Results.Ok`, `Results.NotFound` o `Results.Problem`, explica el código y el cuerpo enviados.

Por qué se hace. El ejemplo PUT reemplaza los campos editables de una orden ya existente, conservando los valores que controla el servidor. No debe inventar una nueva orden cuando el identificador no existe: en nuestro contrato responde 404. Cuando una orden ya está finalizada la regla del dominio impide editarla y se comunica un conflicto 409. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `PUT /{id} → 204`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.2.15 — Comprobar que PATCH no modifica los demás campos

Resultado que vas a construir. Comprobar que PATCH no modifica los demás campos. Este paso corresponde a la etapa `2.2-recursos-verbos-codigos` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `src/AceriaApp.Api/Program.cs` y localiza el concepto `MapPut`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. El ejemplo PUT reemplaza los campos editables de una orden ya existente, conservando los valores que controla el servidor. No debe inventar una nueva orden cuando el identificador no existe: en nuestro contrato responde 404. Cuando una orden ya está finalizada la regla del dominio impide editarla y se comunica un conflicto 409. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `PATCH /{id}/cantidad → 200`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.2.16 — Programar MapDelete para eliminar una orden

Resultado que vas a construir. Programar MapDelete para eliminar una orden. Este paso corresponde a la etapa `2.2-recursos-verbos-codigos` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Código real que debes escribir y estudiar: `src/AceriaApp.Api/Program.cs`.

```
// INICIO PASO 2.2.5: crear una orden, persistirla, devolver 201 + Location.
app.MapPost("/api/v1/ordenes",async (CrearOrdenRequest request,AceriaDbContext db)=>
{
    if(string.IsNullOrWhiteSpace(request.Cliente)||string.IsNullOrWhiteSpace(request.Material)||
request.Cantidad<=0)
        return Results.BadRequest(new {error="Cliente, material y cantidad son obligatorios."});
    if(request.Cantidad>500)
        return Results.Problem(title:"El taller solo admite lotes de hasta 500.",
            statusCode:StatusCodes.Status422UnprocessableEntity);
    var orden=new
Orden{Cliente=request.Cliente,Material=request.Material,Cantidad=request.Cantidad};
    db.Ordenes.Add(orden);
    await db.SaveChangesAsync();
    return Results.CreatedAtRoute("ObtenerOrdenPorId",new{id=orden.Id},OrdenDto.Desde(orden));
})
.WithName("CrearOrden").WithTags("Ordenes")


;
```

Lectura del código. Examina su declaración, los argumentos que ASP.NET Core enlaza, los servicios que resuelve mediante DI y la decisión que produce el resultado HTTP. Si aparece `SaveChangesAsync`, identifica cuándo se confirma la operación; si aparece `Results.Ok`, `Results.NotFound` o `Results.Problem`, explica el código y el cuerpo enviados.

Por qué se hace. El endpoint PATCH de este curso cambia exclusivamente Cantidad. La elección de una ruta especializada para ese campo evita introducir JSON Patch en un módulo introductorio. El alumno comprobará que Cliente y Material no cambian después de aplicar la modificación parcial. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `DELETE /{id} → 204 / 404`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.2.17 — Verificar 204 seguido de GET 404

Resultado que vas a construir. Verificar 204 seguido de GET 404. Este paso corresponde a la etapa `2.2-recursos-verbos-codigos` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `tests/AceriaApp.Api.Tests/EscrituraTests.cs` y localiza el concepto `MapPatch`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. El endpoint PATCH de este curso cambia exclusivamente Cantidad. La elección de una ruta especializada para ese campo evita introducir JSON Patch en un módulo introductorio. El alumno comprobará que Cliente y Material no cambian después de aplicar la modificación parcial. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `POST /{id}/finalizar → 200 / 409`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.2.18 — Programar la transición de estado finalizar

Resultado que vas a construir. Programar la transición de estado finalizar. Este paso corresponde a la etapa `2.2-recursos-verbos-codigos` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Código real que debes escribir y estudiar: **`src/AceriaApp.Api/Program.cs`**.

```
// INICIO PASO 2.2.5: crear una orden, persistirla, devolver 201 + Location.
app.MapPost("/api/v1/ordenes",async (CrearOrdenRequest request,AceriaDbContext db)=>
{
    if(string.IsNullOrWhiteSpace(request.Cliente)||string.IsNullOrWhiteSpace(request.Material)||
request.Cantidad<=0)
        return Results.BadRequest(new {error="Cliente, material y cantidad son obligatorios."});
    if(request.Cantidad>500)
        return Results.Problem(title:"El taller solo admite lotes de hasta 500.",
            statusCode:StatusCodes.Status422UnprocessableEntity);
    var orden=new
Orden{Cliente=request.Cliente,Material=request.Material,Cantidad=request.Cantidad};
    db.Ordenes.Add(orden);
    await db.SaveChangesAsync();
    return Results.CreatedAtRoute("ObtenerOrdenPorId",new{id=orden.Id},OrdenDto.Desde(orden));
})
.WithName("CrearOrden").WithTags("Ordenes")


;
```

Lectura del código. Examina su declaración, los argumentos que ASP.NET Core enlaza, los servicios que resuelve mediante DI y la decisión que produce el resultado HTTP. Si aparece `SaveChangesAsync`, identifica cuándo se confirma la operación; si aparece `Results.Ok`, `Results.NotFound` o `Results.Problem`, explica el código y el cuerpo enviados.

Por qué se hace. El endpoint PATCH de este curso cambia exclusivamente Cantidad. La elección de una ruta especializada para ese campo evita introducir JSON Patch en un módulo introductorio. El alumno comprobará que Cliente y Material no cambian después de aplicar la modificación parcial. En esta

actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `Cantidad 501 → 422`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.2.19 — Bloquear PUT y PATCH para órdenes finalizadas

Resultado que vas a construir. Bloquear PUT y PATCH para órdenes finalizadas. Este paso corresponde a la etapa `2.2-recursos-verbos-codigos` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `tests/AceriaApp.Api.Tests/EscrituraTests.cs` y localiza el concepto `MapPatch`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. El endpoint PATCH de este curso cambia exclusivamente Cantidad. La elección de una ruta especializada para ese campo evita introducir JSON Patch en un módulo introductorio. El alumno comprobará que Cliente y Material no cambian después de aplicar la modificación parcial. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `POST /api/v1/ordenes → 201 + Location`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.2.20 — Bloquear DELETE y devolver conflicto 409

Resultado que vas a construir. Bloquear DELETE y devolver conflicto 409. Este paso corresponde a la etapa `2.2-recursos-verbos-codigos` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `tests/AceriaApp.Api.Tests/EscrituraTests.cs` y localiza el concepto `MapPatch`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. El endpoint PATCH de este curso cambia exclusivamente Cantidad. La elección de una ruta especializada para ese campo evita introducir JSON Patch en un módulo introductorio. El alumno comprobará que Cliente y Material no cambian después de aplicar la modificación parcial. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `PUT /{id} → 204`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.2.21 — Escribir pruebas HTTP POST y Location

Resultado que vas a construir. Escribir pruebas HTTP POST y Location. Este paso corresponde a la etapa `2.2-recursos-verbos-codigos` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Código real que debes escribir y estudiar: **`tests/AceriaApp.Api.Tests/EscrituraTests.cs`**.

```
using System.Net;
using System.Net.Http.Json;
using AceriaApp.Api.Contratos;
using AceriaApp.Api.Modelo;
using Xunit;
namespace AceriaApp.Api.Tests;
// INICIO PASO 2.2.11: probar POST/GET/PUT/PATCH/DELETE con HTTP.
public class EscrituraTests
{
    [Fact] public async Task CrearYConsultar_Es201Y200()
    {
        using var f=new ApiFactory();using var client=f.CreateClient();
        var created=await client.PostAsJsonAsync("/api/v1/ordenes",
            new CrearOrdenRequest("ACX","CHAPA",10));
        Assert.Equal(HttpStatusCode.Created,created.StatusCode);
        Assert.NotNull(created.Headers.Location);
        var get=await client.GetAsync(created.Headers.Location);
```

```
        Assert.Equal(HttpStatusCode.OK,get.StatusCode);
    }
    [Fact] public async Task EditarYEliminar_CambiaPersistencia()
    {
        using var f=new ApiFactory();using var c=f.CreateClient();
        var made=await c.PostAsJsonAsync("/api/v1/ordenes",
            new CrearOrdenRequest("ACX","CHAPA",10));
        var dto=await made.Content.ReadFromJsonAsync<OrdenDto>();
        Assert.NotNull(dto);
        var put=await c.PutAsJsonAsync("/api/v1/ordenes/"+dto.Id,
            new CrearOrdenRequest("ACX2","CHAPA",11));
        Assert.Equal(HttpStatusCode.NoContent,put.StatusCode);
        var patch=await c.PatchAsJsonAsync("/api/v1/ordenes/"+dto.Id+"/cantidad",
            new ActualizarCantidadRequest(12));
        Assert.Equal(HttpStatusCode.OK,patch.StatusCode);
        var changed=await patch.Content.ReadFromJsonAsync<OrdenDto>();
        Assert.Equal(12,changed?.Cantidad);
        var del=await c.DeleteAsync("/api/v1/ordenes/"+dto.Id);
        Assert.Equal(HttpStatusCode.NoContent,del.StatusCode);
        var gone=await c.GetAsync("/api/v1/ordenes/"+dto.Id);
        Assert.Equal(HttpStatusCode.NotFound,gone.StatusCode);
    }
    [Fact] public async Task ReglaCapacidad_Es422()
    {
        using var f=new ApiFactory();
        var r=await f.CreateClient().PostAsJsonAsync("/api/v1/ordenes",
            new CrearOrdenRequest("ACX","CHAPA",501));
        Assert.Equal(HttpStatusCode.UnprocessableEntity,r.StatusCode);
    }
    [Fact] public async Task OrdenFinalizada_NoSeEdita409()
    {
        using var f=new ApiFactory();using var c=f.CreateClient();
        var made=await c.PostAsJsonAsync("/api/v1/ordenes",
            new CrearOrdenRequest("ACX","CHAPA",2));
        var dto=await made.Content.ReadFromJsonAsync<OrdenDto>();
        Assert.NotNull(dto);
        var done=await c.PostAsync("/api/v1/ordenes/"+dto.Id+"/finalizar",null);
        Assert.Equal(HttpStatusCode.OK,done.StatusCode);
        var put=await c.PutAsJsonAsync("/api/v1/ordenes/"+dto.Id,
            new CrearOrdenRequest("ACX","CHAPA",3));
        Assert.Equal(HttpStatusCode.Conflict,put.StatusCode);
    }
}
```

Lectura del código. Examina su declaración, los argumentos que ASP.NET Core enlaza, los servicios que resuelve mediante DI y la decisión que produce el resultado HTTP. Si aparece `SaveChangesAsync`, identifica cuándo se confirma la operación; si aparece `Results.Ok`, `Results.NotFound` o `Results.Problem`, explica el código y el cuerpo enviados.

Por qué se hace. DELETE busca el recurso y ejecuta Remove y SaveChangesAsync. La primera eliminación devuelve 204; la consulta posterior devuelve 404. La idempotencia describe el estado final del sistema, no la obligación de devolver exactamente el mismo código en peticiones repetidas. Para reproducir 409 se añade la transición POST finalizar y se intenta editar una orden cuyo estado ya no permite modificaciones. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `PATCH /{id}/cantidad → 200`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.2.22 — Escribir pruebas HTTP PUT, PATCH y DELETE

Resultado que vas a construir. Escribir pruebas HTTP PUT, PATCH y DELETE. Este paso corresponde a la etapa `2.2-recursos-verbos-codigos` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Código real que debes escribir y estudiar: `tests/AceriaApp.Api.Tests/EscrituraTests.cs`.

```
using System.Net;
using System.Net.Http.Json;
using AceriaApp.Api.Contratos;
using AceriaApp.Api.Modelo;
using Xunit;
namespace AceriaApp.Api.Tests;
// INICIO PASO 2.2.11: probar POST/GET/PUT/PATCH/DELETE con HTTP.
public class EscrituraTests
{
    [Fact] public async Task CrearYConsultar_Es201Y200()
    {
        using var f=new ApiFactory();using var client=f.CreateClient();
        var created=await client.PostAsJsonAsync("/api/v1/ordenes",
            new CrearOrdenRequest("ACX","CHAPA",10));
        Assert.Equal(HttpStatusCode.Created,created.StatusCode);
        Assert.NotNull(created.Headers.Location);
        var get=await client.GetAsync(created.Headers.Location);
        Assert.Equal(HttpStatusCode.OK,get.StatusCode);
    }
    [Fact] public async Task EditarYEliminar_CambiaPersistencia()
    {
        using var f=new ApiFactory();using var c=f.CreateClient();
        var made=await c.PostAsJsonAsync("/api/v1/ordenes",
            new CrearOrdenRequest("ACX","CHAPA",10));
        var dto=await made.Content.ReadFromJsonAsync<OrdenDto>();
        Assert.NotNull(dto);
        var put=await c.PutAsJsonAsync("/api/v1/ordenes/"+dto.Id,
            new CrearOrdenRequest("ACX2","CHAPA",11));
        Assert.Equal(HttpStatusCode.NoContent,put.StatusCode);
        var patch=await c.PatchAsJsonAsync("/api/v1/ordenes/"+dto.Id+"/cantidad",
            new ActualizarCantidadRequest(12));
        Assert.Equal(HttpStatusCode.OK,patch.StatusCode);
        var changed=await patch.Content.ReadFromJsonAsync<OrdenDto>();
        Assert.Equal(12,changed?.Cantidad);
```

```
        var del=await c.DeleteAsync("/api/v1/ordenes/"+dto.Id);
        Assert.Equal(HttpStatusCode.NoContent,del.StatusCode);
        var gone=await c.GetAsync("/api/v1/ordenes/"+dto.Id);
        Assert.Equal(HttpStatusCode.NotFound,gone.StatusCode);
    }
    [Fact] public async Task ReglaCapacidad_Es422()
    {
        using var f=new ApiFactory();
        var r=await f.CreateClient().PostAsJsonAsync("/api/v1/ordenes",
            new CrearOrdenRequest("ACX","CHAPA",501));
        Assert.Equal(HttpStatusCode.UnprocessableEntity,r.StatusCode);
    }
    [Fact] public async Task OrdenFinalizada_NoSeEdita409()
    {
        using var f=new ApiFactory();using var c=f.CreateClient();
        var made=await c.PostAsJsonAsync("/api/v1/ordenes",
            new CrearOrdenRequest("ACX","CHAPA",2));
        var dto=await made.Content.ReadFromJsonAsync<OrdenDto>();
        Assert.NotNull(dto);
        var done=await c.PostAsync("/api/v1/ordenes/"+dto.Id+"/finalizar",null);
        Assert.Equal(HttpStatusCode.OK,done.StatusCode);
        var put=await c.PutAsJsonAsync("/api/v1/ordenes/"+dto.Id,
            new CrearOrdenRequest("ACX","CHAPA",3));
        Assert.Equal(HttpStatusCode.Conflict,put.StatusCode);
    }
}
```

Lectura del código. Examina su declaración, los argumentos que ASP.NET Core enlaza, los servicios que resuelve mediante DI y la decisión que produce el resultado HTTP. Si aparece `SaveChangesAsync`, identifica cuándo se confirma la operación; si aparece `Results.Ok`, `Results.NotFound` o `Results.Problem`, explica el código y el cuerpo enviados.

Por qué se hace. DELETE busca el recurso y ejecuta Remove y SaveChangesAsync. La primera eliminación devuelve 204; la consulta posterior devuelve 404. La idempotencia describe el estado final del sistema, no la obligación de devolver exactamente el mismo código en peticiones repetidas. Para reproducir 409 se añade la transición POST finalizar y se intenta editar una orden cuyo estado ya no permite modificaciones. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `DELETE /{id} → 204 / 404`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.2.23 — Probar automáticamente 422 y 409

Resultado que vas a construir. Probar automáticamente 422 y 409. Este paso corresponde a la etapa `2.2-recursos-verbos-codigos` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `tests/AceriaApp.Api.Tests/EscrituraTests.cs` y localiza el concepto `MapDelete`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. DELETE busca el recurso y ejecuta Remove y SaveChangesAsync. La primera eliminación devuelve 204; la consulta posterior devuelve 404. La idempotencia describe el estado final del sistema, no la obligación de devolver exactamente el mismo código en peticiones repetidas. Para reproducir 409 se añade la transición POST finalizar y se intenta editar una orden cuyo estado ya no permite modificaciones. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `POST /{id}/finalizar → 200 / 409`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.2.24 — Reto: comprobar idempotencia y efecto persistente

Resultado que vas a construir. Reto: comprobar idempotencia y efecto persistente. Este paso corresponde a la etapa `2.2-recursos-verbos-codigos` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `tests/AceriaApp.Api.Tests/EscrituraTests.cs` y localiza el concepto `MapDelete`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. DELETE busca el recurso y ejecuta Remove y SaveChangesAsync. La primera eliminación devuelve 204; la consulta posterior devuelve 404. La idempotencia describe el estado final del sistema, no la obligación de devolver exactamente el mismo código en peticiones repetidas. Para reproducir 409 se añade la transición POST finalizar y se intenta editar una orden cuyo estado ya no permite modificaciones. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `Cantidad 501 → 422`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

Matriz de comprobación del punto

Comportamiento	Resultado esperado
`POST /api/v1/ordenes`	201 + Location
`PUT /{id}`	204
`PATCH /{id}/cantidad`	200
`DELETE /{id}`	204 / 404
`POST /{id}/finalizar`	200 / 409
`Cantidad 501`	422

Reto integrador. Reconstruye sin consultar la solución el último endpoint o validación practicada y añade una prueba HTTP que demuestre su resultado. Describe el recurso, el verbo, el contrato y el código devuelto. Si no puedes reproducir una respuesta, vuelve al primer paso que introdujo el comportamiento.

Punto 2.3 — Validación efectiva de peticiones con FluentValidation

Proyecto comprobado: `PROYECTO/2.3-validacion-peticiones`. El punto comienza estudiando el contrato de recursos y concluye con una ejecución verificable. Ninguna comprobación sustituye a un endpoint realmente implementado.

PASO 2.3.1 — Comprobar endpoints anteriores y localizar sus contratos

Resultado que vas a construir. Comprobar endpoints anteriores y localizar sus contratos. Este paso corresponde a la etapa `2.3-validacion-peticiones` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `src/AceriaApp.Api/Validacion/Validadores.cs` y localiza el concepto `RuleFor`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. ASP.NET Core comprueba aspectos de la deserialización del JSON, FluentValidation aplica reglas semánticas y el manejador aplica decisiones del dominio. Un cuerpo con cantidad 0 falla la semántica; cantidad 501 satisface el rango del contrato pero supera el límite de negocio de 500. El objetivo es demostrar por qué el primer caso produce 400 y el segundo 422. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `Cliente vacío → 400 + errors`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.3.2 — Distinguir JSON incorrecto, semántica y negocio

Resultado que vas a construir. Distinguir JSON incorrecto, semántica y negocio. Este paso corresponde a la etapa `2.3-validacion-peticiones` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `src/AceriaApp.Api/Validacion/Validadores.cs` y localiza el concepto `RuleFor`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. ASP.NET Core comprueba aspectos de la deserialización del JSON, FluentValidation aplica reglas semánticas y el manejador aplica decisiones del dominio. Un cuerpo con cantidad 0 falla la semántica; cantidad 501 satisface el rango del contrato pero supera el límite de negocio de 500. El objetivo es demostrar por qué el primer caso produce 400 y el segundo 422. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `Cantidad 10001 → 400 + errors`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.3.3 — Instalar FluentValidation.DependencyInjectionExtensions

Resultado que vas a construir. Instalar FluentValidation.DependencyInjectionExtensions. Este paso corresponde a la etapa `2.3-validacion-peticiones` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `src/AceriaApp.Api/Validacion/Validadores.cs` y localiza el concepto `RuleFor`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. ASP.NET Core comprueba aspectos de la deserialización del JSON, FluentValidation aplica reglas semánticas y el manejador aplica decisiones del dominio. Un cuerpo con cantidad 0 falla la semántica; cantidad 501 satisface el rango del contrato pero supera el límite de negocio de 500. El objetivo es demostrar por qué el primer caso produce 400 y el segundo 422. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `Cantidad 501 → 422`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.3.4 — Crear CrearOrdenValidator y reglas de Cliente

Resultado que vas a construir. Crear CrearOrdenValidator y reglas de Cliente. Este paso corresponde a la etapa `2.3-validacion-peticiones` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Código real que debes escribir y estudiar: **`src/AceriaApp.Api/Validacion/Validadores.cs`**.

```
using AceriaApp.Api.Contratos;
using FluentValidation;
namespace AceriaApp.Api.Validacion;
// INICIO PASO 2.3.3
public sealed class CrearOrdenValidator : AbstractValidator<CrearOrdenRequest>
{
    public CrearOrdenValidator()
    {
        RuleFor(x=>x.Cliente).NotEmpty().MaximumLength(100);
        RuleFor(x=>x.Material).NotEmpty().MaximumLength(100);
        RuleFor(x=>x.Cantidad).InclusiveBetween(1,10000);
    }
}
public sealed class ActualizarCantidadValidator : AbstractValidator<ActualizarCantidadRequest>
{
    public ActualizarCantidadValidator() => RuleFor(x=>x.Cantidad).InclusiveBetween(1,10000);
}
// FIN PASO 2.3.3
```

Lectura del código. Examina su declaración, los argumentos que ASP.NET Core enlaza, los servicios que resuelve mediante DI y la decisión que produce el resultado HTTP. Si aparece `SaveChangesAsync`, identifica cuándo se confirma la operación; si aparece `Results.Ok`, `Results.NotFound` o `Results.Problem`, explica el código y el cuerpo enviados.

Por qué se hace. ASP.NET Core comprueba aspectos de la deserialización del JSON, FluentValidation aplica reglas semánticas y el manejador aplica decisiones del dominio. Un cuerpo con cantidad 0 falla la semántica; cantidad 501 satisface el rango del contrato pero supera el límite de negocio de 500. El objetivo es demostrar por qué el primer caso produce 400 y el segundo 422. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `Orden finalizada → 409`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.3.5 — Definir Material y rango válido de Cantidad

Resultado que vas a construir. Definir Material y rango válido de Cantidad. Este paso corresponde a la etapa `2.3-validacion-peticiones` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `src/AceriaApp.Api/Validacion/Validadores.cs` y localiza el concepto `RuleFor`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. ASP.NET Core comprueba aspectos de la deserialización del JSON, FluentValidation aplica reglas semánticas y el manejador aplica decisiones del dominio. Un cuerpo con cantidad 0 falla la semántica; cantidad 501 satisface el rango del contrato pero supera el límite de negocio de 500. El objetivo es demostrar por qué el primer caso produce 400 y el segundo 422. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `Cliente vacío → 400 + errors`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.3.6 — Crear ActualizarCantidadValidator para PATCH

Resultado que vas a construir. Crear ActualizarCantidadValidator para PATCH. Este paso corresponde a la etapa `2.3-validacion-peticiones` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Código real que debes escribir y estudiar: **`src/AceriaApp.Api/Validacion/Validadores.cs`**.

```
using AceriaApp.Api.Contratos;
using FluentValidation;
namespace AceriaApp.Api.Validacion;
// INICIO PASO 2.3.3
public sealed class CrearOrdenValidator : AbstractValidator<CrearOrdenRequest>
{
    public CrearOrdenValidator()
    {
        RuleFor(x=>x.Cliente).NotEmpty().MaximumLength(100);
        RuleFor(x=>x.Material).NotEmpty().MaximumLength(100);
        RuleFor(x=>x.Cantidad).InclusiveBetween(1,10000);
    }
}
public sealed class ActualizarCantidadValidator : AbstractValidator<ActualizarCantidadRequest>
{
    public ActualizarCantidadValidator() => RuleFor(x=>x.Cantidad).InclusiveBetween(1,10000);
}
// FIN PASO 2.3.3
```

Lectura del código. Examina su declaración, los argumentos que ASP.NET Core enlaza, los servicios que resuelve mediante DI y la decisión que produce el resultado HTTP. Si aparece `SaveChangesAsync`, identifica cuándo se confirma la operación; si aparece `Results.Ok`, `Results.NotFound` o `Results.Problem`, explica el código y el cuerpo enviados.

Por qué se hace. CrearOrdenValidator deriva de AbstractValidator y declara reglas NotEmpty, MaximumLength e InclusiveBetween. El validador es código C# ejecutable, no una anotación narrativa. Permite reutilizar reglas en POST y PUT y establecer pruebas de frontera cuando la cantidad es 0, 1, 500, 501 y 10001. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `Cantidad 10001 → 400 + errors`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.3.7 — Ejecutar validadores y estudiar ValidationResult

Resultado que vas a construir. Ejecutar validadores y estudiar ValidationResult. Este paso corresponde a la etapa `2.3-validacion-peticiones` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `src/AceriaApp.Api/Validacion/ValidationFilter.cs` y localiza el concepto `CrearOrdenValidator`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. CrearOrdenValidator deriva de AbstractValidator y declara reglas NotEmpty, MaximumLength e InclusiveBetween. El validador es código C# ejecutable, no una anotación narrativa. Permite reutilizar reglas en POST y PUT y establecer pruebas de frontera cuando la cantidad es 0, 1, 500, 501 y 10001. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `Cantidad 501 → 422`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.3.8 — Crear ValidationFilter genérico de Minimal API

Resultado que vas a construir. Crear ValidationFilter genérico de Minimal API. Este paso corresponde a la etapa `2.3-validacion-peticiones` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Código real que debes escribir y estudiar: **`src/AceriaApp.Api/Validacion/ValidationFilter.cs`**.

```
using FluentValidation;
namespace AceriaApp.Api.Validacion;
// INICIO PASO 2.3.4
public sealed class ValidationFilter<T>(IValidator<T> validator): IEndpointFilter where T:class
{
    public async ValueTask<object?> InvokeAsync(EndpointFilterInvocationContext context,
        EndpointFilterDelegate next)
    {
        var request=context.Arguments.OfType<T>().FirstOrDefault();
        if(request is null) return Results.ValidationProblem(
            new Dictionary<string,string[]> { ["body"]=new[]{"Falta el cuerpo."} });
        var result=await validator.ValidateAsync(request,context.HttpContext.RequestAborted);
        if(!result.IsValid)
        {
            var errors=result.Errors.GroupBy(x=>x.PropertyName)
                .ToDictionary(x=>x.Key,x=>x.Select(e=>e.ErrorMessage).ToArray());
            return Results.ValidationProblem(errors);
```

```
        }
        return await next(context);
    }
}
// FIN PASO 2.3.4
```

Lectura del código. Examina su declaración, los argumentos que ASP.NET Core enlaza, los servicios que resuelve mediante DI y la decisión que produce el resultado HTTP. Si aparece `SaveChangesAsync`, identifica cuándo se confirma la operación; si aparece `Results.Ok`, `Results.NotFound` o `Results.Problem`, explica el código y el cuerpo enviados.

Por qué se hace. CrearOrdenValidator deriva de AbstractValidator y declara reglas NotEmpty, MaximumLength e InclusiveBetween. El validador es código C# ejecutable, no una anotación narrativa. Permite reutilizar reglas en POST y PUT y establecer pruebas de frontera cuando la cantidad es 0, 1, 500, 501 y 10001. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `Orden finalizada → 409`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.3.9 — Obtener la petición de EndpointFilterInvocationContext

Resultado que vas a construir. Obtener la petición de EndpointFilterInvocationContext. Este paso corresponde a la etapa `2.3-validacion-peticiones` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `src/AceriaApp.Api/Validacion/ValidationFilter.cs` y localiza el concepto `CrearOrdenValidator`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. CrearOrdenValidator deriva de AbstractValidator y declara reglas NotEmpty, MaximumLength e InclusiveBetween. El validador es código C# ejecutable, no una anotación narrativa. Permite reutilizar reglas en POST y PUT y establecer pruebas de frontera cuando la cantidad es 0, 1, 500, 501 y 10001. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `Cliente vacío → 400 + errors`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.3.10 — Ejecutar ValidateAsync y respetar RequestAborted

Resultado que vas a construir. Ejecutar ValidateAsync y respetar RequestAborted. Este paso corresponde a la etapa `2.3-validacion-peticiones` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `src/AceriaApp.Api/Validacion/ValidationFilter.cs` y localiza el concepto `CrearOrdenValidator`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. CrearOrdenValidator deriva de AbstractValidator y declara reglas NotEmpty, MaximumLength e InclusiveBetween. El validador es código C# ejecutable, no una anotación narrativa. Permite reutilizar reglas en POST y PUT y establecer pruebas de frontera cuando la cantidad es 0, 1, 500, 501 y 10001. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `Cantidad 10001 → 400 + errors`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.3.11 — Agrupar errores por campo con LINQ

Resultado que vas a construir. Agrupar errores por campo con LINQ. Este paso corresponde a la etapa `2.3-validacion-peticiones` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Código real que debes escribir y estudiar: **`src/AceriaApp.Api/Validacion/ValidationFilter.cs`**.

```
using FluentValidation;
namespace AceriaApp.Api.Validacion;
// INICIO PASO 2.3.4
public sealed class ValidationFilter<T>(IValidator<T> validator): IEndpointFilter where T:class
```

```
{
    public async ValueTask<object?> InvokeAsync(EndpointFilterInvocationContext context,
        EndpointFilterDelegate next)
    {
        var request=context.Arguments.OfType<T>().FirstOrDefault();
        if(request is null) return Results.ValidationProblem(
            new Dictionary<string,string[]> { ["body"]=new[]{"Falta el cuerpo."} });
        var result=await validator.ValidateAsync(request,context.HttpContext.RequestAborted);
        if(!result.IsValid)
        {
            var errors=result.Errors.GroupBy(x=>x.PropertyName)
                .ToDictionary(x=>x.Key,x=>x.Select(e=>e.ErrorMessage).ToArray());
            return Results.ValidationProblem(errors);
        }
        return await next(context);
    }
}
// FIN PASO 2.3.4
```

Lectura del código. Examina su declaración, los argumentos que ASP.NET Core enlaza, los servicios que resuelve mediante DI y la decisión que produce el resultado HTTP. Si aparece `SaveChangesAsync`, identifica cuándo se confirma la operación; si aparece `Results.Ok`, `Results.NotFound` o `Results.Problem`, explica el código y el cuerpo enviados.

Por qué se hace. ValidationFilter recibe el argumento de la petición, ejecuta ValidateAsync y agrupa errores por propiedad. Si hay fallos devuelve Results.ValidationProblem con el campo errors; solo cuando IsValid es verdadero continúa mediante next(context). Omitir esa condición permitiría procesar datos rechazados. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `Cantidad 501 → 422`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.3.12 — Responder mediante Results.ValidationProblem

Resultado que vas a construir. Responder mediante Results.ValidationProblem. Este paso corresponde a la etapa `2.3-validacion-peticiones` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `src/AceriaApp.Api/Validacion/ValidationFilter.cs` y localiza el concepto `InvokeAsync`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. ValidationFilter recibe el argumento de la petición, ejecuta ValidateAsync y agrupa errores por propiedad. Si hay fallos devuelve Results.ValidationProblem con el campo errors; solo cuando IsValid es verdadero continúa mediante next(context). Omitir esa condición permitiría procesar datos rechazados. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `Orden finalizada → 409`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.3.13 — Permitir next(context) solo cuando IsValid

Resultado que vas a construir. Permitir next(context) solo cuando IsValid. Este paso corresponde a la etapa `2.3-validacion-peticiones` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `src/AceriaApp.Api/Program.cs` y localiza el concepto `InvokeAsync`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. ValidationFilter recibe el argumento de la petición, ejecuta ValidateAsync y agrupa errores por propiedad. Si hay fallos devuelve Results.ValidationProblem con el campo errors; solo cuando IsValid es verdadero continúa mediante next(context). Omitir esa condición permitiría procesar datos rechazados. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `Cliente vacío → 400 + errors`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.3.14 — Registrar validadores en el contenedor DI

Resultado que vas a construir. Registrar validadores en el contenedor DI. Este paso corresponde a la etapa `2.3-validacion-peticiones` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Código real que debes escribir y estudiar: `src/AceriaApp.Api/Program.cs`.

```
using AceriaApp.Api.Datos;
using AceriaApp.Api.Modelo;
using Microsoft.EntityFrameworkCore;
using AceriaApp.Api.Contratos;
using AceriaApp.Api.Validacion;
using FluentValidation;

var builder=WebApplication.CreateBuilder(args);
```

Lectura del código. Examina su declaración, los argumentos que ASP.NET Core enlaza, los servicios que resuelve mediante DI y la decisión que produce el resultado HTTP. Si aparece `SaveChangesAsync`, identifica cuándo se confirma la operación; si aparece `Results.Ok`, `Results.NotFound` o `Results.Problem`, explica el código y el cuerpo enviados.

Por qué se hace. ValidationFilter recibe el argumento de la petición, ejecuta ValidateAsync y agrupa errores por propiedad. Si hay fallos devuelve Results.ValidationProblem con el campo errors; solo cuando IsValid es verdadero continúa mediante next(context). Omitir esa condición permitiría procesar datos rechazados. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `Cantidad 10001 → 400 + errors`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.3.15 — Conectar filtro de CrearOrdenRequest al POST

Resultado que vas a construir. Conectar filtro de CrearOrdenRequest al POST. Este paso corresponde a la etapa `2.3-validacion-peticiones` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Código real que debes escribir y estudiar: `src/AceriaApp.Api/Program.cs`.

```
using AceriaApp.Api.Datos;
using AceriaApp.Api.Modelo;
using Microsoft.EntityFrameworkCore;
using AceriaApp.Api.Contratos;
```

```
using AceriaApp.Api.Validacion;
using FluentValidation;

var builder=WebApplication.CreateBuilder(args);
```

Lectura del código. Examina su declaración, los argumentos que ASP.NET Core enlaza, los servicios que resuelve mediante DI y la decisión que produce el resultado HTTP. Si aparece `SaveChangesAsync`, identifica cuándo se confirma la operación; si aparece `Results.Ok`, `Results.NotFound` o `Results.Problem`, explica el código y el cuerpo enviados.

Por qué se hace. ValidationFilter recibe el argumento de la petición, ejecuta ValidateAsync y agrupa errores por propiedad. Si hay fallos devuelve Results.ValidationProblem con el campo errors; solo cuando IsValid es verdadero continúa mediante next(context). Omitir esa condición permitiría procesar datos rechazados. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `Cantidad 501 → 422`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.3.16 — Conectar filtro de CrearOrdenRequest al PUT

Resultado que vas a construir. Conectar filtro de CrearOrdenRequest al PUT. Este paso corresponde a la etapa `2.3-validacion-peticiones` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `src/AceriaApp.Api/Program.cs` y localiza el concepto `AddEndpointFilter`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. AddValidatorsFromAssemblyContaining registra servicios en el contenedor, pero no aplica reglas de manera automática a un endpoint. Para ejecutar la validación se utiliza AddEndpointFilter en POST, PUT y PATCH con el contrato correspondiente. Los tests deben comprobar el comportamiento HTTP, no limitarse a instanciar un validador. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `Orden finalizada → 409`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.3.17 — Conectar filtro de ActualizarCantidadRequest al PATCH

Resultado que vas a construir. Conectar filtro de ActualizarCantidadRequest al PATCH. Este paso corresponde a la etapa `2.3-validacion-peticiones` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Código real que debes escribir y estudiar: **`src/AceriaApp.Api/Program.cs`**.

```
using AceriaApp.Api.Datos;
using AceriaApp.Api.Modelo;
using Microsoft.EntityFrameworkCore;
using AceriaApp.Api.Contratos;
using AceriaApp.Api.Validacion;
using FluentValidation;

var builder=WebApplication.CreateBuilder(args);
```

Lectura del código. Examina su declaración, los argumentos que ASP.NET Core enlaza, los servicios que resuelve mediante DI y la decisión que produce el resultado HTTP. Si aparece `SaveChangesAsync`, identifica cuándo se confirma la operación; si aparece `Results.Ok`, `Results.NotFound` o `Results.Problem`, explica el código y el cuerpo enviados.

Por qué se hace. AddValidatorsFromAssemblyContaining registra servicios en el contenedor, pero no aplica reglas de manera automática a un endpoint. Para ejecutar la validación se utiliza AddEndpointFilter en POST, PUT y PATCH con el contrato correspondiente. Los tests deben comprobar el comportamiento HTTP, no limitarse a instanciar un validador. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `Cliente vacío → 400 + errors`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.3.18 — Enviar POST incorrecto y comprobar status 400

Resultado que vas a construir. Enviar POST incorrecto y comprobar status 400. Este paso corresponde a la etapa `2.3-validacion-peticiones` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `src/AceriaApp.Api/Program.cs` y localiza el concepto `AddEndpointFilter`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. AddValidatorsFromAssemblyContaining registra servicios en el contenedor, pero no aplica reglas de manera automática a un endpoint. Para ejecutar la validación se utiliza AddEndpointFilter en POST, PUT y PATCH con el contrato correspondiente. Los tests deben comprobar el comportamiento HTTP, no limitarse a instanciar un validador. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `Cantidad 10001 → 400 + errors`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.3.19 — Inspeccionar la propiedad errors del Problem Details

Resultado que vas a construir. Inspeccionar la propiedad errors del Problem Details. Este paso corresponde a la etapa `2.3-validacion-peticiones` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `tests/AceriaApp.Api.Tests/ValidacionHttpTests.cs` y localiza el concepto `AddEndpointFilter`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. AddValidatorsFromAssemblyContaining registra servicios en el contenedor, pero no aplica reglas de manera automática a un endpoint. Para ejecutar la validación se utiliza AddEndpointFilter en POST, PUT y PATCH con el contrato correspondiente. Los tests deben comprobar el comportamiento HTTP, no limitarse a instanciar un validador. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `Cantidad 501 → 422`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.3.20 — Probar límite 10001 frente a la capacidad 501

Resultado que vas a construir. Probar límite 10001 frente a la capacidad 501. Este paso corresponde a la etapa `2.3-validacion-peticiones` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `tests/AceriaApp.Api.Tests/ValidacionHttpTests.cs` y localiza el concepto `AddEndpointFilter`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. AddValidatorsFromAssemblyContaining registra servicios en el contenedor, pero no aplica reglas de manera automática a un endpoint. Para ejecutar la validación se utiliza AddEndpointFilter en POST, PUT y PATCH con el contrato correspondiente. Los tests deben comprobar el comportamiento HTTP, no limitarse a instanciar un validador. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `Orden finalizada → 409`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.3.21 — Reproducir 409 con orden ya finalizada

Resultado que vas a construir. Reproducir 409 con orden ya finalizada. Este paso corresponde a la etapa `2.3-validacion-peticiones` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `tests/AceriaApp.Api.Tests/ValidacionHttpTests.cs` y localiza el concepto `Results.Problem`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. Un error de validación 400 debe ofrecer mensajes por propiedad. Una capacidad de taller incumplida se documenta en el contrato como 422. Intentar modificar una orden finalizada genera un 409 porque depende del estado persistido. La diferencia permite a los clientes automatizar el

tratamiento de errores. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `Cliente vacío → 400 + errors`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.3.22 — Crear pruebas de validación mediante HTTP

Resultado que vas a construir. Crear pruebas de validación mediante HTTP. Este paso corresponde a la etapa `2.3-validacion-peticiones` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Código real que debes escribir y estudiar: `tests/AceriaApp.Api.Tests/ValidacionHttpTests.cs`.

```
using System.Net;
using System.Net.Http.Json;
using System.Text.Json;
using AceriaApp.Api.Contratos;
using Xunit;
namespace AceriaApp.Api.Tests;
// INICIO PASO 2.3.8: se verifica el filtro ACTIVO, no su mera existencia.
public class ValidacionHttpTests
{
    [Theory]
    [InlineData("","CHAPA",10)]
    [InlineData("ACX","",10)]
    [InlineData("ACX","CHAPA",0)]
    public async Task DatosInvalidos_Es400ConErrors(string cliente,string material,int cantidad)
    {
        using var f=new ApiFactory();
        var r=await f.CreateClient().PostAsJsonAsync("/api/v1/ordenes",
            new CrearOrdenRequest(cliente,material,cantidad));
        Assert.Equal(HttpStatusCode.BadRequest,r.StatusCode);
        using var json=JsonDocument.Parse(await r.Content.ReadAsStringAsync());
        Assert.True(json.RootElement.TryGetProperty("errors",out _));
    }
    [Fact] public async Task CantidadMasDe10000_Es400()
    {
        using var f=new ApiFactory();
        var r=await f.CreateClient().PostAsJsonAsync("/api/v1/ordenes",
            new CrearOrdenRequest("ACX","CHAPA",10001));
        Assert.Equal(HttpStatusCode.BadRequest,r.StatusCode);
    }
    [Fact] public async Task ReglaDeNegocio_501_Es422()
```

```
    {
        using var f=new ApiFactory();
        var r=await f.CreateClient().PostAsJsonAsync("/api/v1/ordenes",
            new CrearOrdenRequest("ACX","CHAPA",501));
        Assert.Equal(HttpStatusCode.UnprocessableEntity,r.StatusCode);
    }
}
```

Lectura del código. Examina su declaración, los argumentos que ASP.NET Core enlaza, los servicios que resuelve mediante DI y la decisión que produce el resultado HTTP. Si aparece `SaveChangesAsync`, identifica cuándo se confirma la operación; si aparece `Results.Ok`, `Results.NotFound` o `Results.Problem`, explica el código y el cuerpo enviados.

Por qué se hace. Un error de validación 400 debe ofrecer mensajes por propiedad. Una capacidad de taller incumplida se documenta en el contrato como 422. Intentar modificar una orden finalizada genera un 409 porque depende del estado persistido. La diferencia permite a los clientes automatizar el tratamiento de errores. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `Cantidad 10001 → 400 + errors`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.3.23 — Verificar que los GET siguen sin regresiones

Resultado que vas a construir. Verificar que los GET siguen sin regresiones. Este paso corresponde a la etapa `2.3-validacion-peticiones` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `tests/AceriaApp.Api.Tests/ValidacionHttpTests.cs` y localiza el concepto `Results.Problem`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. Un error de validación 400 debe ofrecer mensajes por propiedad. Una capacidad de taller incumplida se documenta en el contrato como 422. Intentar modificar una orden finalizada genera un 409 porque depende del estado persistido. La diferencia permite a los clientes automatizar el tratamiento de errores. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `Cantidad 501 → 422`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.3.24 — Reto: personalizar mensajes y comprobar errores

Resultado que vas a construir. Reto: personalizar mensajes y comprobar errores. Este paso corresponde a la etapa `2.3-validacion-peticiones` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `tests/AceriaApp.Api.Tests/ValidacionHttpTests.cs` y localiza el concepto `Results.Problem`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. Un error de validación 400 debe ofrecer mensajes por propiedad. Una capacidad de taller incumplida se documenta en el contrato como 422. Intentar modificar una orden finalizada genera un 409 porque depende del estado persistido. La diferencia permite a los clientes automatizar el tratamiento de errores. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `Orden finalizada → 409`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

Matriz de comprobación del punto

Comportamiento	Resultado esperado
`Cliente vacío`	400 + errors
`Cantidad 10001`	400 + errors
`Cantidad 501`	422
`Orden finalizada`	409

Reto integrador. Reconstruye sin consultar la solución el último endpoint o validación practicada y añade una prueba HTTP que demuestre su resultado. Describe el recurso, el verbo, el contrato y el código devuelto. Si no puedes reproducir una respuesta, vuelve al primer paso que introdujo el comportamiento.

Punto 2.4 — Documento OpenAPI generado y comprobado con Swagger

Proyecto comprobado: `PROYECTO/2.4-documentacion-openapi`. El punto comienza estudiando el contrato de recursos y concluye con una ejecución verificable. Ninguna comprobación sustituye a un endpoint realmente implementado.

PASO 2.4.1 — Comprobar el contrato REST que se va a documentar

Resultado que vas a construir. Comprobar el contrato REST que se va a documentar. Este paso corresponde a la etapa `2.4-documentacion-openapi` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `src/AceriaApp.Api/Documentacion/DocumentacionExtensions.cs` y localiza el concepto `AddSwaggerGen`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. OpenAPI no es una cadena YAML fabricada para satisfacer un test superficial. Swashbuckle descubre los endpoints realmente declarados en ASP.NET Core y genera el documento JSON. AddEndpointsApiExplorer registra la exploración de rutas y SwaggerDoc aporta metadatos de la versión v1. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `GET /swagger/v1/swagger.json → 200 en Development`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.4.2 — Instalar Swashbuckle.AspNetCore compatible con .NET 8

Resultado que vas a construir. Instalar Swashbuckle.AspNetCore compatible con .NET 8. Este paso corresponde a la etapa `2.4-documentacion-openapi` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `src/AceriaApp.Api/Documentacion/DocumentacionExtensions.cs` y localiza el concepto `AddSwaggerGen`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. OpenAPI no es una cadena YAML fabricada para satisfacer un test superficial. Swashbuckle descubre los endpoints realmente declarados en ASP.NET Core y genera el documento JSON. AddEndpointsApiExplorer registra la exploración de rutas y SwaggerDoc aporta metadatos de la versión v1. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `info.version → v1`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.4.3 — Crear carpeta Documentacion y extensiones de configuración

Resultado que vas a construir. Crear carpeta Documentacion y extensiones de configuración. Este paso corresponde a la etapa `2.4-documentacion-openapi` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Código real que debes escribir y estudiar: `src/AceriaApp.Api/Documentacion/DocumentacionExtensions.cs`.

```
using AceriaApp.Api.Contratos;
using Microsoft.OpenApi.Any;
using Microsoft.OpenApi.Models;
using Swashbuckle.AspNetCore.SwaggerGen;
namespace AceriaApp.Api.Documentacion;
// INICIO PASO 2.4.3
public static class DocumentacionExtensions
{
    public static IServiceCollection AddDocumentacion(this IServiceCollection services)
    {
        services.AddEndpointsApiExplorer();
        services.AddSwaggerGen(options=>
        {
            options.SwaggerDoc("v1",new OpenApiInfo{
                Title="AceriaApp — Órdenes de producción",Version="v1",
                Description="API REST educativa creada con ASP.NET Core 8.",
                Contact=new OpenApiContact{Name="Equipo AceriaApp",Email="equipo@acerialocal"}
            });
            options.SchemaFilter<EjemploCrearOrden>();
        });
        return services;
    }
    public static WebApplication UseDocumentacion(this WebApplication app)
    {
        if(app.Environment.IsDevelopment())
        {
```

```
            app.UseSwagger();
            app.UseSwaggerUI(o=>o.SwaggerEndpoint("/swagger/v1/swagger.json","AceriaApp v1"));
        }
        return app;
    }
}
// FIN PASO 2.4.3
// INICIO PASO 2.4.5
public sealed class EjemploCrearOrden : ISchemaFilter
{
    public void Apply(OpenApiSchema schema,SchemaFilterContext context)
    {
        if(context.Type!=typeof(CrearOrdenRequest)) return;
        schema.Example=new OpenApiObject{
            ["cliente"]=new OpenApiString("ACX-001"),
            ["material"]=new OpenApiString("CHAPA-3MM"),
            ["cantidad"]=new OpenApiInteger(10)
        };
    }
}
// FIN PASO 2.4.5
```

Lectura del código. Examina su declaración, los argumentos que ASP.NET Core enlaza, los servicios que resuelve mediante DI y la decisión que produce el resultado HTTP. Si aparece `SaveChangesAsync`, identifica cuándo se confirma la operación; si aparece `Results.Ok`, `Results.NotFound` o `Results.Problem`, explica el código y el cuerpo enviados.

Por qué se hace. OpenAPI no es una cadena YAML fabricada para satisfacer un test superficial. Swashbuckle descubre los endpoints realmente declarados en ASP.NET Core y genera el documento JSON. AddEndpointsApiExplorer registra la exploración de rutas y SwaggerDoc aporta metadatos de la versión v1. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `paths contiene GET y POST → Rutas descubiertas`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.4.4 — Registrar AddEndpointsApiExplorer y AddSwaggerGen

Resultado que vas a construir. Registrar AddEndpointsApiExplorer y AddSwaggerGen. Este paso corresponde a la etapa `2.4-documentacion-openapi` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Código real que debes escribir y estudiar: **`src/AceriaApp.Api/Documentacion/DocumentacionExtensions.cs`**.

```
using AceriaApp.Api.Contratos;
using Microsoft.OpenApi.Any;
using Microsoft.OpenApi.Models;
using Swashbuckle.AspNetCore.SwaggerGen;
namespace AceriaApp.Api.Documentacion;
// INICIO PASO 2.4.3
public static class DocumentacionExtensions
{
    public static IServiceCollection AddDocumentacion(this IServiceCollection services)
    {
        services.AddEndpointsApiExplorer();
        services.AddSwaggerGen(options=>
        {
            options.SwaggerDoc("v1",new OpenApiInfo{
                Title="AceriaApp — Órdenes de producción",Version="v1",
                Description="API REST educativa creada con ASP.NET Core 8.",
                Contact=new OpenApiContact{Name="Equipo AceriaApp",Email="equipo@acerialocal"}
            });
            options.SchemaFilter<EjemploCrearOrden>();
        });
        return services;
    }
    public static WebApplication UseDocumentacion(this WebApplication app)
    {
        if(app.Environment.IsDevelopment())
        {
            app.UseSwagger();
            app.UseSwaggerUI(o=>o.SwaggerEndpoint("/swagger/v1/swagger.json","AceriaApp v1"));
        }
        return app;
    }
}
// FIN PASO 2.4.3
// INICIO PASO 2.4.5
public sealed class EjemploCrearOrden : ISchemaFilter
{
    public void Apply(OpenApiSchema schema,SchemaFilterContext context)
    {
        if(context.Type!=typeof(CrearOrdenRequest)) return;
        schema.Example=new OpenApiObject{
            ["cliente"]=new OpenApiString("ACX-001"),
            ["material"]=new OpenApiString("CHAPA-3MM"),
            ["cantidad"]=new OpenApiInteger(10)
        };
    }
}
// FIN PASO 2.4.5
```

Lectura del código. Examina su declaración, los argumentos que ASP.NET Core enlaza, los servicios que resuelve mediante DI y la decisión que produce el resultado HTTP. Si aparece `SaveChangesAsync`, identifica cuándo se confirma la operación; si aparece `Results.Ok`, `Results.NotFound` o `Results.Problem`, explica el código y el cuerpo enviados.

Por qué se hace. OpenAPI no es una cadena YAML fabricada para satisfacer un test superficial. Swashbuckle descubre los endpoints realmente declarados en ASP.NET Core y genera el documento JSON. AddEndpointsApiExplorer registra la exploración de rutas y SwaggerDoc aporta metadatos de la versión v1. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `schemas contiene CrearOrdenRequest → Contrato registrado`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.4.5 — Configurar SwaggerDoc con información y versión v1

Resultado que vas a construir. Configurar SwaggerDoc con información y versión v1. Este paso corresponde a la etapa `2.4-documentacion-openapi` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `src/AceriaApp.Api/Documentacion/DocumentacionExtensions.cs` y localiza el concepto `AddSwaggerGen`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. OpenAPI no es una cadena YAML fabricada para satisfacer un test superficial. Swashbuckle descubre los endpoints realmente declarados en ASP.NET Core y genera el documento JSON. AddEndpointsApiExplorer registra la exploración de rutas y SwaggerDoc aporta metadatos de la versión v1. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `GET /swagger/v1/swagger.json → 404 en Production`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.4.6 — Conectar AddDocumentacion en Program.cs

Resultado que vas a construir. Conectar AddDocumentacion en Program.cs. Este paso corresponde a la etapa `2.4-documentacion-openapi` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Código real que debes escribir y estudiar: **`src/AceriaApp.Api/Program.cs`**.

```
using AceriaApp.Api.Datos;
using AceriaApp.Api.Modelo;
using Microsoft.EntityFrameworkCore;
using AceriaApp.Api.Contratos;
using AceriaApp.Api.Validacion;
using FluentValidation;
using AceriaApp.Api.Documentacion;

var builder=WebApplication.CreateBuilder(args);
```

Lectura del código. Examina su declaración, los argumentos que ASP.NET Core enlaza, los servicios que resuelve mediante DI y la decisión que produce el resultado HTTP. Si aparece `SaveChangesAsync`, identifica cuándo se confirma la operación; si aparece `Results.Ok`, `Results.NotFound` o `Results.Problem`, explica el código y el cuerpo enviados.

Por qué se hace. La aplicación debe ejecutar AddDocumentacion antes de builder.Build y UseDocumentacion después. El primer método registra servicios y el segundo habilita el middleware. Definir una clase de extensión que nunca se invoca en Program.cs no documenta nada. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `GET /swagger/v1/swagger.json → 200 en Development`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.4.7 — Configurar UseSwagger y Swagger UI en Development

Resultado que vas a construir. Configurar UseSwagger y Swagger UI en Development. Este paso corresponde a la etapa `2.4-documentacion-openapi` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `src/AceriaApp.Api/Documentacion/DocumentacionExtensions.cs` y localiza el concepto `AddDocumentacion`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. La aplicación debe ejecutar AddDocumentacion antes de builder.Build y UseDocumentacion después. El primer método registra servicios y el segundo habilita el middleware. Definir una clase de extensión que nunca se invoca en Program.cs no documenta nada. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `info.version → v1`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.4.8 — Conectar UseDocumentacion y abrir /swagger

Resultado que vas a construir. Conectar UseDocumentacion y abrir /swagger. Este paso corresponde a la etapa `2.4-documentacion-openapi` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Código real que debes escribir y estudiar: `src/AceriaApp.Api/Program.cs`.

```
using AceriaApp.Api.Datos;
using AceriaApp.Api.Modelo;
using Microsoft.EntityFrameworkCore;
using AceriaApp.Api.Contratos;
using AceriaApp.Api.Validacion;
using FluentValidation;
using AceriaApp.Api.Documentacion;

var builder=WebApplication.CreateBuilder(args);
```

Lectura del código. Examina su declaración, los argumentos que ASP.NET Core enlaza, los servicios que resuelve mediante DI y la decisión que produce el resultado HTTP. Si aparece `SaveChangesAsync`, identifica cuándo se confirma la operación; si aparece `Results.Ok`, `Results.NotFound` o `Results.Problem`, explica el código y el cuerpo enviados.

Por qué se hace. La aplicación debe ejecutar AddDocumentacion antes de builder.Build y UseDocumentacion después. El primer método registra servicios y el segundo habilita el middleware. Definir una clase de extensión que nunca se invoca en Program.cs no documenta nada. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `paths contiene GET y POST → Rutas descubiertas`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.4.9 — Descargar el JSON real /swagger/v1/swagger.json

Resultado que vas a construir. Descargar el JSON real /swagger/v1/swagger.json. Este paso corresponde a la etapa `2.4-documentacion-openapi` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `src/AceriaApp.Api/Program.cs` y localiza el concepto `AddDocumentacion`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. La aplicación debe ejecutar AddDocumentacion antes de builder.Build y UseDocumentacion después. El primer método registra servicios y el segundo habilita el middleware. Definir una clase de extensión que nunca se invoca en Program.cs no documenta nada. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `schemas contiene CrearOrdenRequest → Contrato registrado`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.4.10 — Identificar info, paths, components y schemas

Resultado que vas a construir. Identificar info, paths, components y schemas. Este paso corresponde a la etapa `2.4-documentacion-openapi` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `src/AceriaApp.Api/Program.cs` y localiza el concepto `AddDocumentacion`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. La aplicación debe ejecutar AddDocumentacion antes de builder.Build y UseDocumentacion después. El primer método registra servicios y el segundo habilita el middleware. Definir una clase de extensión que nunca se invoca en Program.cs no documenta nada. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `GET /swagger/v1/swagger.json → 404 en Production`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.4.11 — Aplicar WithName y WithTags a los endpoints

Resultado que vas a construir. Aplicar WithName y WithTags a los endpoints. Este paso corresponde a la etapa `2.4-documentacion-openapi` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `src/AceriaApp.Api/Program.cs` y localiza el concepto `Produces`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. WithName, WithSummary, WithTags, Produces, ProducesProblem y ProducesValidationProblem describen operaciones reales. Declarar que un endpoint puede devolver 400 no hace que valide: la validación efectiva ya se incorporó en 2.3. La documentación debe corresponder a lo que el runtime responde. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `GET /swagger/v1/swagger.json → 200 en Development`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.4.12 — Documentar GET con WithSummary y Produces

Resultado que vas a construir. Documentar GET con WithSummary y Produces. Este paso corresponde a la etapa `2.4-documentacion-openapi` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Código real que debes escribir y estudiar: `src/AceriaApp.Api/Program.cs`.

```
using AceriaApp.Api.Datos;
using AceriaApp.Api.Modelo;
using Microsoft.EntityFrameworkCore;
using AceriaApp.Api.Contratos;
using AceriaApp.Api.Validacion;
using FluentValidation;
using AceriaApp.Api.Documentacion;

var builder=WebApplication.CreateBuilder(args);
```

Lectura del código. Examina su declaración, los argumentos que ASP.NET Core enlaza, los servicios que resuelve mediante DI y la decisión que produce el resultado HTTP. Si aparece `SaveChangesAsync`, identifica cuándo se confirma la operación; si aparece `Results.Ok`, `Results.NotFound` o `Results.Problem`, explica el código y el cuerpo enviados.

Por qué se hace. WithName, WithSummary, WithTags, Produces, ProducesProblem y ProducesValidationProblem describen operaciones reales. Declarar que un endpoint puede devolver 400 no hace que valide: la validación efectiva ya se incorporó en 2.3. La documentación debe corresponder a lo que el runtime responde. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `info.version → v1`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.4.13 — Documentar POST con 201 y ValidationProblem 400

Resultado que vas a construir. Documentar POST con 201 y ValidationProblem 400. Este paso corresponde a la etapa `2.4-documentacion-openapi` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `src/AceriaApp.Api/Program.cs` y localiza el concepto `Produces`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. WithName, WithSummary, WithTags, Produces, ProducesProblem y ProducesValidationProblem describen operaciones reales. Declarar que un endpoint puede devolver 400 no hace que valide: la validación efectiva ya se incorporó en 2.3. La documentación debe corresponder a lo que el runtime responde. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `paths contiene GET y POST → Rutas descubiertas`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.4.14 — Documentar PUT y PATCH con códigos alternativos

Resultado que vas a construir. Documentar PUT y PATCH con códigos alternativos. Este paso corresponde a la etapa `2.4-documentacion-openapi` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `src/AceriaApp.Api/Program.cs` y localiza el concepto `Produces`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. WithName, WithSummary, WithTags, Produces, ProducesProblem y ProducesValidationProblem describen operaciones reales. Declarar que un endpoint puede devolver 400 no hace que valide: la validación efectiva ya se incorporó en 2.3. La documentación debe corresponder a lo que el runtime responde. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `schemas contiene CrearOrdenRequest → Contrato registrado`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.4.15 — Documentar DELETE y el conflicto 409

Resultado que vas a construir. Documentar DELETE y el conflicto 409. Este paso corresponde a la etapa `2.4-documentacion-openapi` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `src/AceriaApp.Api/Program.cs` y localiza el concepto `Produces`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. WithName, WithSummary, WithTags, Produces, ProducesProblem y ProducesValidationProblem describen operaciones reales. Declarar que un endpoint puede devolver 400 no hace que valide: la validación efectiva ya se incorporó en 2.3. La documentación debe corresponder a lo que el runtime responde. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `GET /swagger/v1/swagger.json → 404 en Production`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.4.16 — Crear SchemaFilter EjemploCrearOrden

Resultado que vas a construir. Crear SchemaFilter EjemploCrearOrden. Este paso corresponde a la etapa `2.4-documentacion-openapi` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Código real que debes escribir y estudiar: `src/AceriaApp.Api/Documentacion/DocumentacionExtensions.cs`.

```
using AceriaApp.Api.Contratos;
using Microsoft.OpenApi.Any;
using Microsoft.OpenApi.Models;
using Swashbuckle.AspNetCore.SwaggerGen;
namespace AceriaApp.Api.Documentacion;
// INICIO PASO 2.4.3
public static class DocumentacionExtensions
{
    public static IServiceCollection AddDocumentacion(this IServiceCollection services)
    {
        services.AddEndpointsApiExplorer();
        services.AddSwaggerGen(options=>
        {
            options.SwaggerDoc("v1",new OpenApiInfo{
                Title="AceriaApp — Órdenes de producción",Version="v1",
                Description="API REST educativa creada con ASP.NET Core 8.",
                Contact=new OpenApiContact{Name="Equipo AceriaApp",Email="equipo@acerialocal"}
            });
            options.SchemaFilter<EjemploCrearOrden>();
        });
        return services;
    }
```

```
    public static WebApplication UseDocumentacion(this WebApplication app)
    {
        if(app.Environment.IsDevelopment())
        {
            app.UseSwagger();
            app.UseSwaggerUI(o=>o.SwaggerEndpoint("/swagger/v1/swagger.json","AceriaApp v1"));
        }
        return app;
    }
}
// FIN PASO 2.4.3
// INICIO PASO 2.4.5
public sealed class EjemploCrearOrden : ISchemaFilter
{
    public void Apply(OpenApiSchema schema,SchemaFilterContext context)
    {
        if(context.Type!=typeof(CrearOrdenRequest)) return;
        schema.Example=new OpenApiObject{
            ["cliente"]=new OpenApiString("ACX-001"),
            ["material"]=new OpenApiString("CHAPA-3MM"),
            ["cantidad"]=new OpenApiInteger(10)
        };
    }
}
// FIN PASO 2.4.5
```

Lectura del código. Examina su declaración, los argumentos que ASP.NET Core enlaza, los servicios que resuelve mediante DI y la decisión que produce el resultado HTTP. Si aparece `SaveChangesAsync`, identifica cuándo se confirma la operación; si aparece `Results.Ok`, `Results.NotFound` o `Results.Problem`, explica el código y el cuerpo enviados.

Por qué se hace. SchemaFilter permite mostrar en components.schemas un ejemplo de CrearOrdenRequest con cliente, material y cantidad. El ejemplo no crea una fila en SQL Server; explica el cuerpo que puede enviar el consumidor. El DTO de salida, en cambio, describe la representación de una orden ya existente. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `GET /swagger/v1/swagger.json → 200 en Development`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.4.17 — Inspeccionar example de CrearOrdenRequest en Swagger

Resultado que vas a construir. Inspeccionar example de CrearOrdenRequest en Swagger. Este paso corresponde a la etapa `2.4-documentacion-openapi` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `tests/AceriaApp.Api.Tests/ContratoOpenApiTests.cs` y localiza el concepto `EjemploCrearOrden`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. SchemaFilter permite mostrar en components.schemas un ejemplo de CrearOrdenRequest con cliente, material y cantidad. El ejemplo no crea una fila en SQL Server; explica el cuerpo que puede enviar el consumidor. El DTO de salida, en cambio, describe la representación de una orden ya existente. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `info.version → v1`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.4.18 — Ejecutar un POST desde Swagger UI

Resultado que vas a construir. Ejecutar un POST desde Swagger UI. Este paso corresponde a la etapa `2.4-documentacion-openapi` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `tests/AceriaApp.Api.Tests/ContratoOpenApiTests.cs` y localiza el concepto `EjemploCrearOrden`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. SchemaFilter permite mostrar en components.schemas un ejemplo de CrearOrdenRequest con cliente, material y cantidad. El ejemplo no crea una fila en SQL Server; explica el cuerpo que puede enviar el consumidor. El DTO de salida, en cambio, describe la representación de una orden ya existente. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `paths contiene GET y POST → Rutas descubiertas`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.4.19 — Consultar la orden creada desde la propia UI

Resultado que vas a construir. Consultar la orden creada desde la propia UI. Este paso corresponde a la etapa `2.4-documentacion-openapi` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `tests/AceriaApp.Api.Tests/ContratoOpenApiTests.cs` y localiza el concepto `EjemploCrearOrden`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. SchemaFilter permite mostrar en components.schemas un ejemplo de CrearOrdenRequest con cliente, material y cantidad. El ejemplo no crea una fila en SQL Server; explica el cuerpo que puede enviar el consumidor. El DTO de salida, en cambio, describe la representación de una orden ya existente. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `schemas contiene CrearOrdenRequest → Contrato registrado`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.4.20 — Crear tests HTTP del documento OpenAPI real

Resultado que vas a construir. Crear tests HTTP del documento OpenAPI real. Este paso corresponde a la etapa `2.4-documentacion-openapi` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Código real que debes escribir y estudiar: `tests/AceriaApp.Api.Tests/ContratoOpenApiTests.cs`.

```
using System.Net;
using System.Text.Json;
using Microsoft.AspNetCore.Hosting;
using Xunit;
namespace AceriaApp.Api.Tests;
// INICIO PASO 2.4.8: abrir OpenAPI JSON REAL mediante HTTP.
public class ContratoOpenApiTests
{
    [Fact] public async Task OpenApi_ContieneRutasYVersion()
    {
        using var f=new ApiFactory();
```

```
        var r=await f.CreateClient().GetAsync("/swagger/v1/swagger.json");
        Assert.Equal(HttpStatusCode.OK,r.StatusCode);
        using var json=JsonDocument.Parse(await r.Content.ReadAsStringAsync());

Assert.Equal("v1",json.RootElement.GetProperty("info").GetProperty("version").GetString());
        var paths=json.RootElement.GetProperty("paths");
        Assert.True(paths.TryGetProperty("/api/v1/ordenes",out var resource));
        Assert.True(resource.TryGetProperty("get",out _));
        Assert.True(resource.TryGetProperty("post",out _));
    }
    [Fact] public async Task OpenApi_IncluyeEjemplo()
    {
        using var f=new ApiFactory();
        var r=await f.CreateClient().GetAsync("/swagger/v1/swagger.json");
        using var json=JsonDocument.Parse(await r.Content.ReadAsStringAsync());
        var schemas=json.RootElement.GetProperty("components").GetProperty("schemas");
        Assert.True(schemas.TryGetProperty("CrearOrdenRequest",out var schema));
        Assert.True(schema.TryGetProperty("example",out _));
    }
    [Fact] public async Task Produccion_NoExponeSwagger()
    {
        using var f=new ApiFactory();
        using var other=f.WithWebHostBuilder(b=>b.UseEnvironment("Production"));
        var r=await other.CreateClient().GetAsync("/swagger/v1/swagger.json");
        Assert.Equal(HttpStatusCode.NotFound,r.StatusCode);
    }
}
```

Lectura del código. Examina su declaración, los argumentos que ASP.NET Core enlaza, los servicios que resuelve mediante DI y la decisión que produce el resultado HTTP. Si aparece `SaveChangesAsync`, identifica cuándo se confirma la operación; si aparece `Results.Ok`, `Results.NotFound` o `Results.Problem`, explica el código y el cuerpo enviados.

Por qué se hace. SchemaFilter permite mostrar en components.schemas un ejemplo de CrearOrdenRequest con cliente, material y cantidad. El ejemplo no crea una fila en SQL Server; explica el cuerpo que puede enviar el consumidor. El DTO de salida, en cambio, describe la representación de una orden ya existente. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `GET /swagger/v1/swagger.json → 404 en Production`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.4.21 — Comprobar rutas GET y POST en paths

Resultado que vas a construir. Comprobar rutas GET y POST en paths. Este paso corresponde a la etapa `2.4-documentacion-openapi` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `tests/AceriaApp.Api.Tests/ContratoOpenApiTests.cs` y localiza el concepto `UseDocumentacion`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. Swagger UI facilita explorar las operaciones y ejecutarlas desde el navegador. WebApplicationFactory descarga /swagger/v1/swagger.json y contrasta versión, paths y schemas. Otra prueba establece Production y exige que el endpoint de documentación no se publique. Así se comprueba el documento generado y no una cadena aislada del servidor. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `GET /swagger/v1/swagger.json → 200 en Development`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.4.22 — Comprobar schema y ejemplo de petición

Resultado que vas a construir. Comprobar schema y ejemplo de petición. Este paso corresponde a la etapa `2.4-documentacion-openapi` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `tests/AceriaApp.Api.Tests/ContratoOpenApiTests.cs` y localiza el concepto `UseDocumentacion`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. Swagger UI facilita explorar las operaciones y ejecutarlas desde el navegador. WebApplicationFactory descarga /swagger/v1/swagger.json y contrasta versión, paths y schemas. Otra prueba establece Production y exige que el endpoint de documentación no se publique. Así se comprueba el documento generado y no una cadena aislada del servidor. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `info.version → v1`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.4.23 — Comprobar que Production responde 404 para Swagger

Resultado que vas a construir. Comprobar que Production responde 404 para Swagger. Este paso corresponde a la etapa `2.4-documentacion-openapi` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `tests/AceriaApp.Api.Tests/ContratoOpenApiTests.cs` y localiza el concepto `UseDocumentacion`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. Swagger UI facilita explorar las operaciones y ejecutarlas desde el navegador. WebApplicationFactory descarga /swagger/v1/swagger.json y contrasta versión, paths y schemas. Otra prueba establece Production y exige que el endpoint de documentación no se publique. Así se comprueba el documento generado y no una cadena aislada del servidor. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `paths contiene GET y POST → Rutas descubiertas`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

PASO 2.4.24 — Reto: documentar PATCH y verificar su summary en JSON

Resultado que vas a construir. Reto: documentar PATCH y verificar su summary en JSON. Este paso corresponde a la etapa `2.4-documentacion-openapi` de la solución. Identifica en el diseño qué recurso se consulta o se modifica, cuál es el verbo HTTP y qué comportamiento debe poder observar el cliente.

Acciones en Visual Studio. Abre la solución de la etapa en Visual Studio, localiza el proyecto `AceriaApp.Api`, crea el archivo necesario o modifica `Program.cs` cuando corresponda. Escribe cada declaración en su lugar: registra servicios antes de `builder.Build()` y declara los endpoints después. Guarda los cambios y compila con Ctrl+Shift+B. Compara el resultado con los archivos de referencia solo después de intentar construirlo tú mismo.

Trazabilidad con código. Abre `tests/AceriaApp.Api.Tests/ContratoOpenApiTests.cs` y localiza el concepto `UseDocumentacion`. No escribas una clase auxiliar para representar una operación que ya ejecuta ASP.NET Core: modifica el endpoint o la prueba indicada por el título de este paso.

Por qué se hace. Swagger UI facilita explorar las operaciones y ejecutarlas desde el navegador. WebApplicationFactory descarga /swagger/v1/swagger.json y contrasta versión, paths y schemas. Otra prueba establece Production y exige que el endpoint de documentación no se publique. Así se

comprueba el documento generado y no una cadena aislada del servidor. En esta actividad lo importante no es generar un documento administrativo, sino relacionar un elemento del contrato con la ejecución de la API de órdenes y comprender por qué el servidor acepta, rechaza o transforma la petición.

Comprobación mediante HTTP. Como referencia, observa: `schemas contiene CrearOrdenRequest → Contrato registrado`. Si este paso prepara una clase o un servicio, verifica primero que compila y realiza después la petición del paso que lo utiliza. Si modifica un endpoint, envía la petición ahora, inspecciona el código de respuesta y comprueba el JSON o la persistencia de la orden. Al terminar cada punto ejecuta `dotnet test` sobre el proyecto de tests de esa etapa.

Errores frecuentes y cómo resolverlos. Si aparece 404 comprueba la URI, el GUID y el verbo. Si aparece 400 revisa el JSON de entrada y las reglas del contrato. Si recibes 500 inspecciona la excepción en Visual Studio y la configuración de LocalDB o DI. Si el resultado parece correcto pero la base no cambia, localiza `SaveChangesAsync` y verifica que se ejecuta realmente. No avances con un error pendiente.

Matriz de comprobación del punto

Comportamiento	Resultado esperado
`GET /swagger/v1/swagger.json`	200 en Development
`info.version`	v1
`paths contiene GET y POST`	Rutas descubiertas
`schemas contiene CrearOrdenRequest`	Contrato registrado
`GET /swagger/v1/swagger.json`	404 en Production

Reto integrador. Reconstruye sin consultar la solución el último endpoint o validación practicada y añade una prueba HTTP que demuestre su resultado. Describe el recurso, el verbo, el contrato y el código devuelto. Si no puedes reproducir una respuesta, vuelve al primer paso que introdujo el comportamiento.
